Outline

2.1 Introduction

In this chapter, we introduce Python programming and present examples illustrating key language features. We assume you've read the IPython Test-Drive in Chapter 1, which introduced the IPython interpreter and used it to evaluate simple arithmetic expressions.

2.2 Variables and Assignment Statements

You've used IPython's interactive mode as a calculator with expressions such as

```
In [1]: 45 + 72
Out[1]: 117
```

As in algebra, Python expressions also may contain **variables**, which store values for later use in your code. Let's create a variable named `x` that stores the integer `7`, which is the variable's **value**:

```
In [2]: x = 7
```

Snippet `[2]` is a **statement**. Each statement specifies a task to perform. The preceding statement creates `x` and uses the **assignment symbol** (`=`) to give `x` a value. The entire statement is an **assignment statement** that we read as "`x` is assigned the value `7`." Most statements stop at the end of the line, though it's possible for statements to span more than one line. The following statement creates the variable `y` and assigns to it the value `3`:

```
In [3]: y = 3
```

Adding Variable Values and Viewing the Result

You can now use the values of `x` and `y` in expressions:

```
In [4]: x + y
Out[4]: 10
```

The `+` symbol is the **addition operator**. It's a **binary operator** because it has *two* **operands** (in this case, the variables `x` and `y`) on which it performs its operation.

Calculations in Assignment Statements

You'll often save calculation results for later use. The following assignment statement adds the values of variables `x` and `y` and assigns the result to the variable `total`, which we then display:

```
In [5]: total = x + y

In [6]: total
Out[6]: 10
```

Snippet [5] is read, "total is assigned the value of x + y." The = symbol is not an operator. The right side of the = symbol always executes first, then the result is assigned to the variable on the symbol's left side.

Python Style

The *Style Guide for Python Code*[1] helps you write code that conforms to Python's coding conventions. The style guide recommends inserting one space on each side of the assignment symbol = and binary operators like + to make programs more readable.

Variable Names

A variable name, such as x, is an **identifier**. Each identifier may consist of letters, digits and underscores (_) but may not begin with a digit. Python is **case sensitive**, so number and Number are *different* identifiers because one begins with a lowercase letter and the other begins with an uppercase letter.

Types

Each value in Python has a **type** that indicates the kind of data the value represents. You can view a value's type, as in:

```
In [7]: type(x)
Out[7]: int

In [8]: type(10.5)
Out[8]: float
```

The variable x contains the integer value 7 (from snippet [2]), so Python displays int (short for integer). The value 10.5 is a **floating-point number** (that is, a number with a decimal point), so Python displays float.

Python's **type built-in function** determines a value's type. A **function** performs a task when you **call** it by writing its name, followed by parentheses, (). The parentheses contain the function's **argument**—the data that the type function needs to perform its task. You'll create *custom* functions in later chapters.

Self Check

1 *(True/False)* The following are valid variable names: 3g, 87 and score_4.
Answer: False. Because they begin with a digit, 3g and 87 are invalid names.

2 *(True/False)* Python treats y and Y as the same identifier.
Answer: False. Python is case sensitive, so y and Y are different identifiers.

3 *(IPython Session)* Calculate the sum of 10.8, 12.2 and 0.2, store it in the variable total, then display total's value.
Answer:

```
In [1]: total = 10.8 + 12.2 + 0.2

In [2]: total
Out[2]: 23.1
```

1. https://www.python.org/dev/peps/pep-0008/.

2.3 Arithmetic

Many programs perform arithmetic calculations. The following table summarizes the **arithmetic operators**, which include some symbols not used in algebra.

Python operation	Arithmetic operator	Algebraic expression	Python expression
Addition	`+`	$f + 7$	`f + 7`
Subtraction	`-`	$p - c$	`p - c`
Multiplication	`*`	$b \cdot m$	`b * m`
Exponentiation	`**`	x^y	`x ** y`
True division	`/`	x/y or $\frac{x}{y}$ or $x \div y$	`x / y`
Floor division	`//`	$\lfloor x/y \rfloor$ or $\left\lfloor \frac{x}{y} \right\rfloor$ or $\lfloor x \div y \rfloor$	`x // y`
Remainder (modulo)	`%`	$r \bmod s$	`r % s`

Multiplication (*)

Rather than algebra's center dot (·), Python uses the **asterisk (*) multiplication operator**:

```
In [1]: 7 * 4
Out[1]: 28
```

Exponentiation (**)

The **exponentiation (**) operator** raises one value to the power of another:

```
In [2]: 2 ** 10
Out[2]: 1024
```

To calculate the square root, you can use the exponent `1/2` (that is, `0.5`):

```
In [3]: 9 ** (1 / 2)
Out[3]: 3.0
```

True Division (/) vs. Floor Division (//)

True division (/) divides a numerator by a denominator and yields a floating-point number with a decimal point, as in:

```
In [4]: 7 / 4
Out[4]: 1.75
```

Floor division (//) divides a numerator by a denominator, yielding the highest *integer* that's not greater than the result. Python **truncates** (discards) the fractional part:

```
In [5]: 7 // 4
Out[5]: 1

In [6]: 3 // 5
Out[6]: 0

In [7]: 14 // 7
Out[7]: 2
```

In true division, -13 divided by 4 gives -3.25:

```
In [8]: -13 / 4
Out[8]: -3.25
```

Floor division gives the closest integer that's *not greater than* -3.25—which is -4:

```
In [9]: -13 // 4
Out[9]: -4
```

Exceptions and Tracebacks

Dividing by zero with / or // is not allowed and results in an **exception**—a sign that a problem occurred:

```
In [10]: 123 / 0
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
<ipython-input-10-cd759d3fcf39> in <module>()
----> 1 123 / 0

ZeroDivisionError: division by zero
```

Python reports an exception with a **traceback**. This traceback indicates that an exception of type ZeroDivisionError occurred—most exception names end with Error. In interactive mode, the snippet number that caused the exception is specified by the 10 in the line

```
<ipython-input-10-cd759d3fcf39> in <module>()
```

The line that begins with ----> 1 shows the code that caused the exception. Sometimes snippets have more than one line of code—the 1 to the right of ----> indicates that line 1 within the snippet caused the exception. The last line shows the exception that occurred, followed by a colon (:) and an error message with more information about the exception:

```
ZeroDivisionError: division by zero
```

The "Files and Exceptions" chapter discusses exceptions in detail.

An exception also occurs if you try to use a variable that you have not yet created. The following snippet tries to add 7 to the undefined variable z, resulting in a NameError:

```
In [11]: z + 7
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-11-f2cdbf4fe75d> in <module>()
----> 1 z + 7

NameError: name 'z' is not defined
```

Remainder Operator

Python's **remainder operator (%)** yields the remainder after the left operand is divided by the right operand:

```
In [12]: 17 % 5
Out[12]: 2
```

In this case, 17 divided by 5 yields a quotient of 3 and a remainder of 2. This operator is most commonly used with integers, but also can be used with other numeric types:

```
In [13]: 7.5 % 3.5
Out[13]: 0.5
```

In the exercises, we use the remainder operator for applications such as determining whether one number is a multiple of another—a special case of this is determining whether a number is odd or even.

Straight-Line Form

Algebraic notations such as

$$\frac{a}{b}$$

generally are not acceptable to compilers or interpreters. For this reason, algebraic expressions must be typed in **straight-line form** using Python's operators. The expression above must be written as `a / b` (or `a // b` for floor division) so that all operators and operands appear in a horizontal straight line.

Grouping Expressions with Parentheses

Parentheses group Python expressions, as they do in algebraic expressions. For example, the following code multiplies `10` times the quantity `5 + 3`:

```
In [14]: 10 * (5 + 3)
Out[14]: 80
```

Without these parentheses, the result is *different*:

```
In [15]: 10 * 5 + 3
Out[15]: 53
```

The parentheses are **redundant** (unnecessary) if removing them yields the *same* result.

Operator Precedence Rules

Python applies the operators in arithmetic expressions according to the following **rules of operator precedence**. These are generally the same as those in algebra:

1. Expressions in parentheses evaluate first, so parentheses may force the order of evaluation to occur in any sequence you desire. Parentheses have the highest level of precedence. In expressions with **nested parentheses**, such as `(a / (b - c))`, the expression in the *innermost* parentheses (that is, `b - c`) evaluates first.
2. Exponentiation operations evaluate next. If an expression contains several exponentiation operations, Python applies them from right to left.
3. Multiplication, division and modulus operations evaluate next. If an expression contains several multiplication, true-division, floor-division and modulus operations, Python applies them from left to right. Multiplication, division and modulus are "on the same level of precedence."
4. Addition and subtraction operations evaluate last. If an expression contains several addition and subtraction operations, Python applies them from left to right. Addition and subtraction also have the same level of precedence.

We'll expand these rules as we introduce other operators. For the complete list of operators and their precedence (in lowest-to-highest order), see

```
https://docs.python.org/3/reference/expressions.html#operator-
   precedence
```

Operator Grouping

When we say that Python applies certain operators from left to right, we are referring to the operators' **grouping**. For example, in the expression

```
a + b + c
```

the addition operators (+) group from left to right as if we parenthesized the expression as `(a + b) + c`. All Python operators of the same precedence group left-to-right except for the exponentiation operator (**), which groups right-to-left.

Redundant Parentheses

You can use redundant parentheses to group subexpressions to make the expression clearer. For example, the second-degree polynomial

```
y = a * x ** 2 + b * x + c
```

can be parenthesized, for clarity, as

```
y = (a * (x ** 2)) + (b * x) + c
```

Breaking a complex expression into a sequence of statements with shorter, simpler expressions also can promote clarity.

Operand Types

Each arithmetic operator may be used with integers and floating-point numbers. If both operands are integers, the result is an integer—except for the true-division (/) operator, which always yields a floating-point number. If both operands are floating-point numbers, the result is a floating-point number. Expressions containing an integer and a floating-point number are **mixed-type expressions**—these always produce floating-point numbers.

Self Check

1 *(Multiple Choice)* Given that $y = ax^3 + 7$, which of the following is not a correct statement for this equation?

a) `y = a * x * x * x + 7`
b) `y = a * x ** 3 + 7`
c) `y = a * (x * x * x) + 7`
d) `y = a * x * (x * x + 7)`

Answer: d is incorrect.

2 *(True/False)* In nested parentheses, the expression in the innermost pair evaluates last.
Answer: False. The expression in the innermost parentheses evaluates first.

3 *(IPython Session)* Evaluate the expression `3 * (4 - 5)` with and without parentheses. Are the parentheses redundant?
Answer:

```
In [1]: 3 * (4 - 5)
Out[1]: -3

In [2]: 3 * 4 - 5
Out[2]: 7
```

The parentheses are not redundant—if you remove them the resulting value is different.

4 *(IPython Session)* Evaluate the expressions `4 ** 3 ** 2`, `(4 ** 3) ** 2` and `4 ** (3 ** 2)`. Are any of the parentheses redundant?
Answer:

```
In [3]: 4 ** 3 ** 2
Out[3]: 262144

In [4]: (4 ** 3) ** 2
Out[4]: 4096

In [5]: 4 ** (3 ** 2)
Out[5]: 262144
```

Only the parentheses in the last expression are redundant.

2.4 Function `print` and an Intro to Single- and Double-Quoted Strings

The built-in **`print` function** displays its argument(s) as a line of text:

```
In [1]: print('Welcome to Python!')
Welcome to Python!
```

In this case, the argument `'Welcome to Python!'` is a **string**—a sequence of characters enclosed in single quotes (`'`). Unlike when you evaluate expressions in interactive mode, the text that `print` displays here is not preceded by `Out[1]`. Also, `print` does not display a string's quotes, though we'll soon show how to display quotes in strings.

You also may enclose a string in double quotes (`"`), as in:

```
In [2]: print("Welcome to Python!")
Welcome to Python!
```

Python programmers generally prefer single quotes.

When `print` completes its task, it positions the screen cursor at the beginning of the next line. This is similar to what happens when you press the *Enter* (or *Return*) key while typing in a text editor.

Printing a Comma-Separated List of Items

The `print` function can receive a comma-separated list of arguments, as in:

```
In [3]: print('Welcome', 'to', 'Python!')
Welcome to Python!
```

The `print` function displays each argument separated from the next by a space, producing the same output as in the two preceding snippets. Here we showed a comma-separated list of strings, but the values can be of any type. We'll show in the next chapter how to prevent automatic spacing between values or use a different separator than space.

Printing Many Lines of Text with One Statement

When a backslash (`\`) appears in a string, it's known as the **escape character**. The backslash and the character immediately following it form an **escape sequence**. For example, `\n` represents the **newline character** escape sequence, which tells `print` to move the output cursor to the next line. Placing two newline characters back-to-back displays a blank line. The following snippet uses three newline characters to create many lines of output:

```
In [4]: print('Welcome\nto\n\nPython!')
Welcome
to

Python!
```

Other Escape Sequences

The following table shows some common escape sequences.

Escape sequence	Description
\n	Insert a newline character in a string. When the string is displayed, for each newline, move the screen cursor to the beginning of the next line.
\t	Insert a horizontal tab. When the string is displayed, for each tab, move the screen cursor to the next tab stop.
\\	Insert a backslash character in a string.
\"	Insert a double quote character in a string.
\'	Insert a single quote character in a string.

Ignoring a Line Break in a Long String

You may also split a long string (or a long statement) over several lines by using the **\ continuation character** as the last character on a line to ignore the line break:

```
In [5]: print('this is a longer string, so we \
   ...: split it over two lines')
this is a longer string, so we split it over two lines
```

The interpreter reassembles the string's parts into a single string with no line break. Though the backslash character in the preceding snippet is inside a string, it's not the escape character because another character does not follow it.

Printing the Value of an Expression

Calculations can be performed in `print` statements:

```
In [6]: print('Sum is', 7 + 3)
Sum is 10
```

Self Check

1 *(Fill-In)* The _____ function instructs the computer to display information on the screen.

Answer: `print`.

2 *(Fill-In)* Values of the _____ data type contain a sequence of characters.

Answer: string (type `str`).

3 *(IPython Session)* Write an expression that displays the type of `'word'`.

Answer:

```
In [1]: type('word')
Out[1]: str
```

4 *(IPython Session)* What does the following `print` statement display?

```
print('int(5.2)', 'truncates 5.2 to', int(5.2))
```

Answer:

```
In [2]: print('int(5.2)', 'truncates 5.2 to', int(5.2))
int(5.2) truncates 5.2 to 5
```

2.5 Triple-Quoted Strings

Earlier, we introduced strings delimited by a pair of single quotes (') or a pair of double quotes ("). **Triple-quoted strings** begin and end with three double quotes (""") or three single quotes ('''). The *Style Guide for Python Code* recommends three double quotes (""""). Use these to create:

- multiline strings,
- strings containing single or double quotes and
- **docstrings**, which are the recommended way to document the purposes of certain program components.

Including Quotes in Strings

In a string delimited by single quotes, you may include double-quote characters:

```
In [1]: print('Display "hi" in quotes')
Display "hi" in quotes
```

but not single quotes:

```
In [2]: print('Display 'hi' in quotes')
  File "<ipython-input-2-19bf596ccf72>", line 1
    print('Display 'hi' in quotes')
                    ^
SyntaxError: invalid syntax
```

unless you use the \' escape sequence:

```
In [3]: print('Display \'hi\' in quotes')
Display 'hi' in quotes
```

Snippet [2] displayed a **syntax error**, which is a violation of Python's language rules—in this case, a single quote inside a single-quoted string. IPython displays information about the line of code that caused the syntax error and points to the error with a ^ symbol. It also displays the message `SyntaxError: invalid syntax`.

A string delimited by double quotes may include single quote characters:

```
In [4]: print("Display the name O'Brien")
Display the name O'Brien
```

but not double quotes, unless you use the \" escape sequence:

```
In [5]: print("Display \"hi\" in quotes")
Display "hi" in quotes
```

To avoid using \' and \" inside strings, you can enclose such strings in triple quotes:

```
In [6]: print("""Display "hi" and 'bye' in quotes""")
Display "hi" and 'bye' in quotes
```

Multiline Strings

The following snippet assigns a multiline triple-quoted string to `triple_quoted_string`:

```
In [7]: triple_quoted_string = """This is a triple-quoted
   ...: string that spans two lines"""
```

IPython knows that the string is incomplete because we did not type the closing """ before we pressed *Enter*. So, IPython displays a **continuation prompt** `...:` at which you can input the multiline string's next line. This continues until you enter the ending """ and press *Enter*. The following displays `triple_quoted_string`:

```
In [8]: print(triple_quoted_string)
This is a triple-quoted
string that spans two lines
```

Python stores multiline strings with embedded newline escape sequences. When we evaluate `triple_quoted_string` rather than printing it, IPython displays the string in single quotes with a `\n` character where you pressed *Enter* in snippet `[7]`. The quotes IPython displays indicate that `triple_quoted_string` is a string—they're not part of the string's contents:

```
In [9]: triple_quoted_string
Out[9]: 'This is a triple-quoted\nstring that spans two lines'
```

Self Check

1 *(Fill-In)* Multiline strings are enclosed either in _____ or in _____.

Answer: """ (triple double quotes) or ''' (triple single quotes).

2 *(IPython Session)* What displays when you execute the following statement?

```
print("""This is a lengthy
   multiline string containing
a few lines \
of text""")
```

Answer:

```
In [1]: print("""This is a lengthy
   ...:    multiline string containing
   ...: a few lines \
   ...: of text""")
This is a lengthy
   multiline string containing
a few lines of text
```

2.6 Getting Input from the User

The built-in **`input` function** requests and obtains user input:

```
In [1]: name = input("What's your name? ")
What's your name? Paul

In [2]: name
Out[2]: 'Paul'

In [3]: print(name)
Paul
```

The snippet executes as follows:

- First, `input` displays its string argument—called a **prompt**—to tell the user what to type and waits for the user to respond. We typed `Paul` (without quotes) and pressed *Enter*. We use **`bold`** text to distinguish the user's input from the prompt text that `input` displays.
- Function `input` then **returns** (that is, gives back) those characters as a string that the program can use. Here we assigned that string to the variable `name`.

Snippet `[2]` shows `name`'s value. Evaluating `name` displays its value in single quotes as `'Paul'` because it's a string. Printing `name` (in snippet `[3]`) displays the string without the quotes. If you enter quotes, they're part of the string, as in:

```
In [4]: name = input("What's your name? ")
What's your name? 'Paul'

In [5]: name
Out[5]: "'Paul'"

In [6]: print(name)
'Paul'
```

Function `input` Always Returns a String

Consider the following snippets that attempt to read two numbers and add them:

```
In [7]: value1 = input('Enter first number: ')
Enter first number: 7

In [8]: value2 = input('Enter second number: ')
Enter second number: 3

In [9]: value1 + value2
Out[9]: '73'
```

Rather than adding the integers `7` and `3` to produce `10`, Python "adds" the *string* values `'7'` and `'3'`, producing the *string* `'73'`. This is known as **string concatenation**. It creates a new string containing the left operand's value followed by the right operand's value.

Getting an Integer from the User

If you need an integer, convert the string to an integer using the built-in **`int` function**:

```
In [10]: value = input('Enter an integer: ')
Enter an integer: 7

In [11]: value = int(value)

In [12]: value
Out[12]: 7
```

We could have combined the code in snippets `[10]` and `[11]`:

```
In [13]: another_value = int(input('Enter another integer: '))
Enter another integer: 13

In [14]: another_value
Out[14]: 13
```

Variables `value` and `another_value` now contain integers. Adding them produces an integer result (rather than concatenating them):

```
In [15]: value + another_value
Out[15]: 20
```

If the string passed to `int` cannot be converted to an integer, a `ValueError` occurs:

```
In [16]: bad_value = int(input('Enter another integer: '))
Enter another integer: hello
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-16-cd36e6cf8911> in <module>()
----> 1 bad_value = int(input('Enter another integer: '))

ValueError: invalid literal for int() with base 10: 'hello'
```

Function `int` also can convert a floating-point value to an integer:

```
In [17]: int(10.5)
Out[17]: 10
```

To convert strings to floating-point numbers, use the built-in **`float` function**.

Self Check

1 *(Fill-In)* The built-in _____ function converts a floating-point value to an integer value or converts a string representation of an integer to an integer value.
Answer: `int`.

2 *(True/False)* Built-in function `get_input` requests and obtains input from the user.
Answer: False. The built-in function's name is `input`.

3 *(IPython Session)* Use `float` to convert `'6.2'` (a string) to a floating-point value. Multiply that value by `3.3` and show the result.
Answer:

```
In [1]: float('6.2') * 3.3
Out[1]: 20.46
```

2.7 Decision Making: The `if` Statement and Comparison Operators

A **condition** is a Boolean expression with the value **`True`** or **`False`**. The following determines whether `7` is greater than `4` and whether `7` is less than `4`:

```
In [1]: 7 > 4
Out[1]: True

In [2]: 7 < 4
Out[2]: False
```

`True` and `False` are **keywords**—words that Python reserves for its language features. Using a keyword as an identifier causes a `SyntaxError`. `True` and `False` are each capitalized.

You'll often create conditions using the **comparison operators** in the table at the top of the next page:

Algebraic operator	Python operator	Sample condition	Meaning
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Operators >, <, >= and <= all have the same precedence. Operators == and != both have the same precedence, which is lower than that of >, <, >= and <=. A syntax error occurs when any of the operators ==, !=, >= and <= contains spaces between its pair of symbols:

```
In [3]: 7 > = 4
  File "<ipython-input-3-5c6e2897f3b3>", line 1
    7 > = 4
        ^
SyntaxError: invalid syntax
```

Another syntax error occurs if you reverse the symbols in the operators !=, >= and <= (by writing them as =!, => and =<).

Making Decisions with the `if` Statement: Introducing Scripts

We now present a simple version of the **`if` statement**, which uses a condition to decide whether to execute a statement (or a group of statements). Here we'll read two integers from the user and compare them using six consecutive `if` statements, one for each comparison operator. If the condition in a given `if` statement is `True`, the corresponding `print` statement executes; otherwise, it's skipped.

IPython interactive mode is helpful for executing brief code snippets and seeing immediate results. When you have many statements to execute as a group, you typically write them as a **script** stored in a file with the `.py` (short for Python) extension—such as `fig02_01.py` for this example's script. Scripts are also called **programs**. For instructions on locating and executing the scripts in this book, see Chapter 1's IPython Test-Drive.

Each time you execute this script, three of the six conditions are `True`. To show this, we execute the script three times—once with the first integer *less than* the second, once with the *same* value for both integers and once with the first integer *greater than* the second. The three sample executions appear after the script

Figure 2.1 shows the script. Each time we present a script, we introduce it before the figure, then explain the script's code after the figure. We show line numbers for your convenience—these are not part of Python. Integrated development environments (IDEs) enable you to choose whether to display line numbers. To run this example, change to this chapter's `ch02` examples folder, then enter:

```
ipython fig02_01.py
```

or, if you're in IPython already, use the command:

```
run fig02_01.py
```

```
# fig02_01.py
"""Comparing integers using if statements and comparison operators."""

print('Enter two integers, and I will tell you',
      'the relationships they satisfy.')

# read first integer
number1 = int(input('Enter first integer: '))

# read second integer
number2 = int(input('Enter second integer: '))

if number1 == number2:
    print(number1, 'is equal to', number2)

if number1 != number2:
    print(number1, 'is not equal to', number2)

if number1 < number2:
    print(number1, 'is less than', number2)

if number1 > number2:
    print(number1, 'is greater than', number2)

if number1 <= number2:
    print(number1, 'is less than or equal to', number2)

if number1 >= number2:
    print(number1, 'is greater than or equal to', number2)
```

```
Enter two integers and I will tell you the relationships they satisfy.
Enter first integer: 37
Enter second integer: 42
37 is not equal to 42
37 is less than 42
37 is less than or equal to 42
```

```
Enter two integers and I will tell you the relationships they satisfy.
Enter first integer: 7
Enter second integer: 7
7 is equal to 7
7 is less than or equal to 7
7 is greater than or equal to 7
```

```
Enter two integers and I will tell you the relationships they satisfy.
Enter first integer: 54
Enter second integer: 17
54 is not equal to 17
54 is greater than 17
54 is greater than or equal to 17
```

Fig. 2.1 | Comparing integers using `if` statements and comparison operators.

Comments

Line 1 begins with the hash character (#), which indicates that the rest of the line is a **comment**:

```
# fig02_01.py
```

You insert comments to document your code and to improve readability. Comments also help other programmers read and understand your code. They do not cause the computer to perform any action when the code executes. For easy reference, we begin each script with a comment indicating the script's file name.

A comment also can begin to the right of the code on a given line and continue until the end of that line. Such a comment documents the code to its left.

Docstrings

The *Style Guide for Python Code* states that each script should start with a docstring that explains the script's purpose, such as the one in line 2:

```
"""Comparing integers using if statements and comparison operators."""
```

For more complex scripts, the docstring often spans many lines. In later chapters, you'll use docstrings to describe script components you define, such as new functions and new types called classes. We'll also discuss how to access docstrings with the IPython help mechanism.

Blank Lines

Line 3 is a blank line. You use blank lines and space characters to make code easier to read. Together, blank lines, space characters and tab characters are known as **white space**. Python ignores most white space—you'll see that some indentation is required.

Splitting a Lengthy Statement Across Lines

Lines 4–5

```
print('Enter two integers, and I will tell you',
      'the relationships they satisfy.')
```

display instructions to the user. These are too long to fit on one line, so we broke them into two strings. Recall that you can display several values by passing to `print` a comma-separated list—`print` separates each value from the next with a space character.

Typically, you write statements on one line. You may spread a lengthy statement over several lines with the `\` continuation character. Python also allows you to split long code lines in parentheses without using continuation characters (as in lines 4–5). This is the preferred way to break long code lines according to the *Style Guide for Python Code*. Always choose breaking points that make sense, such as after a comma in the preceding call to `print` or before an operator in a lengthy expression.

Reading Integer Values from the User

Next, lines 8 and 11 use the built-in `input` and `int` functions to prompt for and read two integer values from the user.

`if` Statements

The `if` statement in lines 13–14

```
if number1 == number2:
    print(number1, 'is equal to', number2)
```

uses the == comparison operator to determine whether the values of variables `number1` and `number2` are equal. If so, the condition is `True`, and line 14 displays a line of text indicating that the values are equal. If any of the remaining `if` statements' conditions are `True` (lines 16, 19, 22, 25 and 28), the corresponding `print` displays a line of text.

Each `if` statement consists of the keyword `if`, the condition to test, and a colon (`:`) followed by an indented body called a **suite**. Each suite must contain one or more statements. Forgetting the colon (`:`) after the condition is a common syntax error.

Suite Indentation

Python requires you to indent the statements in suites. The *Style Guide for Python Code* recommends four-space indents—we use that convention throughout this book. You'll see in the next chapter that incorrect indentation can cause errors.

Confusing == and =

Using the assignment symbol (=) instead of the equality operator (==) in an `if` statement's condition is a common syntax error. To help avoid this, read == as "is equal to" and = as "is assigned." You'll see in the next chapter that using == in place of = in an assignment statement can lead to subtle problems.

Chaining Comparisons

You can chain comparisons to check whether a value is in a range. The following comparison determines whether `x` is in the range `1` through `5`, inclusive:

```
In [1]: x = 3

In [2]: 1 <= x <= 5
Out[2]: True

In [3]: x = 10

In [4]: 1 <= x <= 5
Out[4]: False
```

Precedence of the Operators We've Presented So Far

The precedence of the operators introduced in this chapter is shown below:

Operators	Grouping	Type
`()`	left to right	parentheses
`**`	right to left	exponentiation
`*  /  //  %`	left to right	multiplication, true division, floor division, remainder
`+  -`	left to right	addition, subtraction
`>  <=  <  >=`	left to right	less than, less than or equal, greater than, greater than or equal
`==  !=`	left to right	equal, not equal

The table lists the operators top-to-bottom in decreasing order of precedence. When writing expressions containing multiple operators, confirm that they evaluate in the order you expect by referring to the operator precedence chart at

```
https://docs.python.org/3/reference/expressions.html#operator-
   precedence
```

Self Check

1 *(Fill-In)* You use _____ to document code and improve its readability.
Answer: comments.

2 *(True/False)* The comparison operators evaluate left to right and all have the same level of precedence.
Answer: False. The operators <, <=, > and >= all have the same level of precedence and evaluate left to right. The operators == and != have the same level of precedence and evaluate left to right. Their precedence is lower than that of <, <=, > and >=.

3 *(IPython Session)* For any of the operators !=, >= or <=, show that a syntax error occurs if you reverse the symbols in a condition.
Answer:

```
In [1]: 7 =< 10
  File "<ipython-input-1-090d4004a38e>", line 1
    7 =< 10
      ^
SyntaxError: invalid syntax
```

4 *(IPython Session)* Use all six comparison operators to compare the values 5 and 9. Display the values on one line using print.
Answer:

```
In [2]: print(5 < 9, 5 <= 9, 5 > 9, 5 >= 9, 5 == 9, 5 != 9)
True True False False False True
```

2.8 Objects and Dynamic Typing

The first chapter introduced the terms *classes* and *objects* and in Section 2.2, we discussed variables, values and types. Values such as 7 (an integer), 4.1 (a floating-point number) and 'dog' are all objects. Every object has a type and a value:

```
In [1]: type(7)
Out[1]: int

In [2]: type(4.1)
Out[2]: float

In [3]: type('dog')
Out[3]: str
```

An object's value is the data stored in the object. The snippets above show objects of Python built-in types **int** (for integers), **float** (for floating-point numbers) and **str** (for strings).

Variables Refer to Objects

Assigning an object to a variable **binds** (associates) that variable's name to the object. As you've seen, you can then use the variable in your code to access the object's value:

```
In [4]: x = 7

In [5]: x + 10
Out[5]: 17

In [6]: x
Out[6]: 7
```

After snippet [4]'s assignment, the variable x **refers** to the integer object containing 7. As shown in snippet [6], snippet [5] does not change x's value. You can change x as follows:

```
In [7]: x = x + 10

In [8]: x
Out[8]: 17
```

Dynamic Typing

Python uses **dynamic typing**—it determines the type of the object a variable refers to while executing your code. We can show this by rebinding the variable x to different objects and checking their types:

```
In [9]: type(x)
Out[9]: int

In [10]: x = 4.1

In [11]: type(x)
Out[11]: float

In [12]: x = 'dog'

In [13]: type(x)
Out[13]: str
```

Garbage Collection

Python creates objects in memory and removes them from memory as necessary. After snippet [10], the variable x now refers to a `float` object. The integer object from snippet [7] is no longer bound to a variable. As we'll discuss in a later chapter, Python automatically removes such objects from memory. This process—called **garbage collection**—helps ensure that memory is available for new objects you create.

Self Check

1 *(Fill-In)* Assigning an object to a variable _____ the variable's name to the object.
Answer: binds.

2 *(True/False)* A variable always references the same object.
Answer: False. You can make an existing variable refer to a different object and even one of a different type.

3 *(IPython Session)* What is the type of the expression `7.5 * 3`?
Answer:

```
In [1]: type(7.5 * 3)
Out[1]: float
```

2.9 Intro to Data Science: Basic Descriptive Statistics

In data science, you'll often use statistics to describe and summarize your data. Here, we begin by introducing several such **descriptive statistics**, including:

- **minimum**—the smallest value in a collection of values.
- **maximum**—the largest value in a collection of values.
- **range**—the range of values from the minimum to the maximum.
- **count**—the number of values in a collection.
- **sum**—the total of the values in a collection.

We'll look at determining the *count* and *sum* in the next chapter. **Measures of dispersion** (also called **measures of variability**), such as *range*, help determine how spread out values are. Other measures of dispersion that we'll present in later chapters include *variance* and *standard deviation*.

Determining the Minimum of Three Values

First, let's show how to determine the minimum of three values manually. The following script prompts for and inputs three values, uses `if` statements to determine the minimum value, then displays it.

```
1  # fig02_02.py
2  """Find the minimum of three values."""
3
4  number1 = int(input('Enter first integer: '))
5  number2 = int(input('Enter second integer: '))
6  number3 = int(input('Enter third integer: '))
7
8  minimum = number1
9
10 if number2 < minimum:
11     minimum = number2
12
13 if number3 < minimum:
14     minimum = number3
15
16 print('Minimum value is', minimum)
```

```
Enter first integer: 12
Enter second integer: 27
Enter third integer: 36
Minimum value is 12
```

```
Enter first integer: 27
Enter second integer: 12
Enter third integer: 36
Minimum value is 12
```

Fig. 2.2 | Find the minimum of three values. (Part 1 of 2.)

```
Enter first integer: 36
Enter second integer: 27
Enter third integer: 12
Minimum value is 12
```

Fig. 2.2 | Find the minimum of three values. (Part 2 of 2.)

After inputting the three values, we process one value at a time:

- First, we assume that `number1` contains the smallest value, so line 8 assigns it to the variable `minimum`. Of course, it's possible that `number2` or `number3` contains the actual smallest value, so we still must compare each of these with `minimum`.
- The first `if` statement (lines 10–11) then tests `number2 < minimum` and if this condition is `True` assigns `number2` to `minimum`.
- The second `if` statement (lines 13–14) then tests `number3 < minimum`, and if this condition is `True` assigns `number3` to `minimum`.

Now, `minimum` contains the smallest value, so we display it. We executed the script three times to show that it always finds the smallest value regardless of whether the user enters it first, second or third.

Determining the Minimum and Maximum with Built-In Functions `min` and `max`

Python has many built-in functions for performing common tasks. Built-in functions **`min`** and **`max`** calculate the minimum and maximum, respectively, of a collection of values:

```
In [1]: min(36, 27, 12)
Out[1]: 12

In [2]: max(36, 27, 12)
Out[2]: 36
```

The functions `min` and `max` can receive any number of arguments.

Determining the Range of a Collection of Values

The *range* of values is simply the minimum through the maximum value. In this case, the range is 12 through 36. Much data science is devoted to getting to know your data. Descriptive statistics is a crucial part of that, but you also have to understand how to interpret the statistics. For example, if you have 100 numbers with a range of 12 through 36, those numbers could be distributed evenly over that range. At the opposite extreme, you could have clumping with 99 values of 12 and one 36, or one 12 and 99 values of 36. In later data science sections, we'll look at common *data distributions*.

Functional-Style Programming: Reduction

Throughout this book, we introduce various *functional-style programming* capabilities. These enable you to write code that can be more concise, clearer and easier to **debug**—that is, find and correct errors. The `min` and `max` functions are examples of a functional-style programming concept called **reduction**. They reduce a collection of values to a *single* value. Other reductions you'll see include the sum, average, variance and standard deviation of a collection of values. You'll also learn how to define custom reductions.

Upcoming Intro to Data Science Sections

In the next two chapters, we'll continue our discussion of basic descriptive statistics with *measures of central tendency*, including *mean*, *median* and *mode*, and *measures of dispersion*, including *variance* and *standard deviation*.

Self Check

1 *(Fill-In)* The range of a collection of values is a measure of _____.
Answer: dispersion.

2 *(IPython Session)* For the values 47, 95, 88, 73, 88 and 84 calculate the minimum, maximum and range.
Answer:

```
In [1]: min(47, 95, 88, 73, 88, 84)
Out[1]: 47

In [2]: max(47, 95, 88, 73, 88, 84)
Out[2]: 95

In [3]: print('Range:', min(47, 95, 88, 73, 88, 84), '-',
   ...:       max(47, 95, 88, 73, 88, 84))
   ...:
Range: 47 - 95
```

2.10 Wrap-Up

This chapter continued our discussion of arithmetic. You used variables to store values for later use. We introduced Python's arithmetic operators and showed that you must write all expressions in straight-line form. You used the built-in function `print` to display data. We created single-, double- and triple-quoted strings. You used triple-quoted strings to create multiline strings and to embed single or double quotes in strings.

You used the `input` function to prompt for and get input from the user at the keyboard. We used the functions `int` and `float` to convert strings to numeric values. We presented Python's comparison operators. Then, you used them in a script that read two integers from the user and compared their values using a series of `if` statements.

We discussed Python's dynamic typing and used the built-in function `type` to display an object's type. Finally, we introduced the basic descriptive statistics minimum and maximum and used them to calculate the range of a collection of values. In the next chapter, you'll learn Python's control statements and program development.

Exercises

Unless specified otherwise, use IPython sessions for each exercise.

2.1 *(What does this code do?)* Create the variables `x = 2` and `y = 3`, then determine what each of the following statements displays:

a) `print('x =', x)`
b) `print('Value of', x, '+', x, 'is', (x + x))`
c) `print('x =')`
d) `print((x + y), '=', (y + x))`

2.2 *(What's wrong with this code?)* The following code should read an integer into the variable `rating`:

```
rating = input('Enter an integer rating between 1 and 10')
```

2.3 *(Fill in the missing code)* Replace *** in the following code with a statement that will print a message like `'Congratulations! Your grade of 91 earns you an A in this course'`. Your statement should print the value stored in the variable `grade`:

```
if grade >= 90:
    ***
```

2.4 *(Arithmetic)* For each of the arithmetic operators `+`, `-`, `*`, `/`, `//` and `**`, display the value of an expression with `27.5` as the left operand and `2` as the right operand.

2.5 *(Circle Area, Diameter and Circumference)* For a circle of radius 2, display the diameter, circumference and area. Use the value 3.14159 for π. Use the following formulas (r is the radius): *diameter* = $2r$, *circumference* = $2\pi r$ and area = πr^2. [In a later chapter, we'll introduce Python's `math` module which contains a higher-precision representation of π.]

2.6 *(Odd or Even)* Use `if` statements to determine whether an integer is odd or even. [*Hint:* Use the remainder operator. An even number is a multiple of 2. Any multiple of 2 leaves a remainder of 0 when divided by 2.]

2.7 *(Multiples)* Use `if` statements to determine whether 1024 is a multiple of 4 and whether 2 is a multiple of 10. (*Hint:* Use the remainder operator.)

2.8 *(Table of Squares and Cubes)* Write a script that calculates the squares and cubes of the numbers from 0 to 5. Print the resulting values in table format, as shown below. Use the tab escape sequence to achieve the three-column output.

```
number  square  cube
0       0       0
1       1       1
2       4       8
3       9       27
4       16      64
5       25      125
```

The next chapter shows how to "right align" numbers. You could try that as an extra challenge here. The output would be:

```
number  square  cube
     0       0     0
     1       1     1
     2       4     8
     3       9    27
     4      16    64
     5      25   125
```

2.9 *(Integer Value of a Character)* Here's a peek ahead. In this chapter, you learned about strings. Each of a string's characters has an integer representation. The set of characters a computer uses together with the characters' integer representations is called that computer's *character set*. You can indicate a character value in a program by enclosing that character in quotes, as in `'A'`. To determine a character's integer value, call the built-in function **`ord`**:

```
In [1]: ord('A')
Out[1]: 65
```

Display the integer equivalents of `B C D b c d 0 1 2 $ * +` and the space character.

2.10 *(Arithmetic, Smallest and Largest)* Write a script that inputs three integers from the user. Display the sum, average, product, smallest and largest of the numbers. Note that each of these is a reduction in functional-style programming.

2.11 *(Separating the Digits in an Integer)* Write a script that inputs a five-digit integer from the user. Separate the number into its individual digits. Print them separated by three spaces each. For example, if the user types in the number `42339`, the script should print

```
4   2   3   3   9
```

Assume that the user enters the correct number of digits. Use both the floor division and remainder operations to "pick off" each digit.

2.12 *(7% Investment Return)* Some investment advisors say that it's reasonable to expect a 7% return over the long term in the stock market. Assuming that you begin with $1000 and leave your money invested, calculate and display how much money you'll have after 10, 20 and 30 years. Use the following formula for determining these amounts:

$$a = p(1 + r)^n$$

where

p is the original amount invested (i.e., the principal of $1000),
r is the annual rate of return (7%),
n is the number of years (10, 20 or 30) and
a is the amount on deposit at the end of the nth year.

2.13 *(How Big Can Python Integers Be?)* We'll answer this question later in the book. For now, use the exponentiation operator `**` with large and very large exponents to produce some huge integers and assign those to the variable `number` to see if Python accepts them. Did you find any integer value that Python won't accept?

2.14 *(Target Heart-Rate Calculator)* While exercising, you can use a heart-rate monitor to see that your heart rate stays within a safe range suggested by your doctors and trainers. According to the American Heart Association (AHA) (`http://bit.ly/AHATargetHeartRates`), the formula for calculating your maximum heart rate in beats per minute is 220 minus your age in years. Your target heart rate is 50–85% of your maximum heart rate. Write a script that prompts for and inputs the user's age and calculates and displays the user's maximum heart rate and the range of the user's target heart rate. [**These formulas are estimates provided by the AHA; maximum and target heart rates may vary based on the health, fitness and gender of the individual. Always consult a physician or qualified healthcare professional before beginning or modifying an exercise program.**]

2.15 *(Sort in Ascending Order)* Write a script that inputs three different floating-point numbers from the user. Display the numbers in increasing order. Recall that an `if` statement's suite can contain more than one statement. Prove that your script works by running it on all six possible orderings of the numbers. Does your script work with duplicate numbers? [This is challenging. In later chapters you'll do this more conveniently and with many more numbers.]

Control Statements and Program Development

3

Objectives

In this chapter, you'll:

- Decide whether to execute actions with the statements `if`, `if`...`else` and `if`...`elif`...`else`.
- Execute statements repeatedly with `while` and `for`.
- Shorten assignment expressions with augmented assignments.
- Use the `for` statement and the built-in `range` function to repeat actions for a sequence of values.
- Perform sentinel-controlled repetition with `while`.
- Learn problem-solving skills: understanding problem requirements, dividing problems into smaller pieces, developing algorithms to solve problems and implementing those algorithms in code.
- Develop algorithms through the process of top-down, stepwise refinement.
- Create compound conditions with the Boolean operators `and`, `or` and `not`.
- Stop looping with `break`.
- Force the next iteration of a loop with `continue`.
- Use some functional-style programming features to write scripts that are more concise, clearer, easier to debug and easier to parallelize.

Outline

3.1 Introduction

Before writing a program to solve a particular problem, you must understand the problem and have a carefully planned approach to solving it. You must also understand Python's building blocks and use proven program-construction principles.

3.2 Algorithms

You can solve any computing problem by executing a series of actions in a specific order. An **algorithm** is a *procedure* for solving a problem in terms of:

1. the **actions** to execute, and
2. the **order** in which these actions execute.

Correctly specifying the order in which the actions execute is essential. Consider the "rise-and-shine algorithm" that an executive follows for getting out of bed and going to work: (1) Get out of bed; (2) take off pajamas; (3) take a shower; (4) get dressed; (5) eat breakfast; (6) carpool to work. This routine gets the executive to work well prepared to make critical decisions. Suppose the executive performs these steps in a different order: (1) Get out of bed; (2) take off pajamas; (3) get dressed; (4) take a shower; (5) eat breakfast; (6) carpool to work. Now, our executive shows up for work soaking wet. **Program control** specifies the order in which statements (actions) execute in a program. This chapter investigates program control using Python's **control statements**.

Self Check

1 *(Fill-In)* A(n) _____ is a procedure for solving a problem. It specifies the _____ to execute and the _____ in which they execute.
Answer: algorithm, actions, order.

3.3 Pseudocode

Pseudocode is an informal English-like language for "thinking out" algorithms. You write text that describes what your program should do. You then convert the pseudocode to Python by replacing pseudocode statements with their Python equivalents.

Addition-Program Pseudocode

The following pseudocode algorithm prompts the user to enter two integers, inputs them from the user at the keyboard, adds them, then stores and displays their sum:

```
Prompt the user to enter the first integer
Input the first integer

Prompt the user to enter the second integer
Input the second integer

Add first integer and second integer, store their sum
Display the numbers and their sum
```

This is the complete pseudocode algorithm. Later in the chapter, we'll show a simple process for creating a pseudocode algorithm from a *requirements statement*. The English pseudocode statements specify the actions you wish to perform and the order in which you wish to perform them.

Self Check

1 ***(True/False)*** Pseudocode is a simple programming language.
Answer: False. Pseudocode is not a programming language. It's an artificial and informal language that helps you develop algorithms.

2 ***(IPython Session)*** Write Python statements that perform the tasks described by this section's pseudocode. Enter the integers `10` and `5`.
Answer:

```
In [1]: number1 = int(input('Enter first integer: '))
Enter first integer: 10

In [2]: number2 = int(input('Enter second integer: '))
Enter second integer: 5

In [3]: total = number1 + number2

In [4]: print('The sum of', number1, 'and', number2, 'is', total)
The sum of 10 and 5 is 15
```

3.4 Control Statements

Usually, statements in a program execute in the order in which they're written. This is called *sequential execution*. Various Python statements enable you to specify that the next statement to execute may be *other than* the next one *in sequence*. This is called *transfer of control* and is achieved with Python *control statements.*

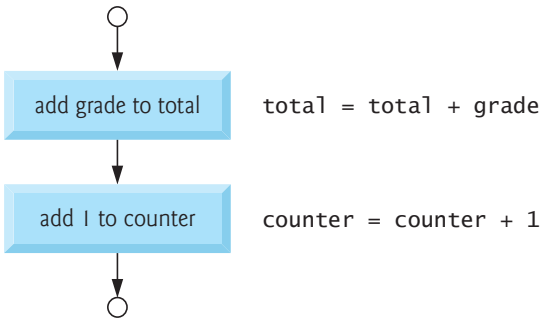
Forms of Control

In the 1960s, extensive use of control transfers was causing difficulty in software development. Blame was pointed at the `goto` statement. This statement allowed you to transfer control to one of many possible destinations in a program. Bohm and Jacopini's research[1] demonstrated that programs could be written without `goto` statements. The notion of *structured programming* became almost synonymous with "goto elimination." Python does not have a `goto` statement. Structured programs are clearer, easier to debug and change, and more likely to be bug-free.

Bohm and Jacopini demonstrated that all programs could be written using three forms of control—namely, **sequential execution**, the **selection statement** and the **repetition statement**. Sequential execution is simple. Python statements execute one after the other "in sequence," unless directed otherwise.

Flowcharts

A **flowchart** is a *graphical* representation of an algorithm or a part of one. You draw flowcharts using *rectangles*, *diamonds*, *rounded rectangles* and *small circles* that you connect by *arrows* called **flowlines**. Like pseudocode, flowcharts are useful for developing and representing algorithms. They clearly show how forms of control operate. Consider the following flowchart segment, which shows *sequential execution*:



We use the **rectangle (or action) symbol** to indicate any *action,* such as a calculation or an input/output operation. The flowlines show the *order* in which the actions execute. First, the grade is added to the total, then 1 is added to the counter. We show the Python code next to each action symbol for comparison purposes. This code is not part of the flowchart.

In a flowchart for a *complete* algorithm, the first symbol is a **rounded rectangle** containing the word "Begin." The last symbol is a rounded rectangle containing the word "End." In a flowchart for only a *part* of an algorithm, we omit the rounded rectangles, instead using small circles called **connector symbols**. The most important symbol is the **decision (or diamond) symbol**, which indicates that a *decision* is to be made, such as in an `if` statement. We begin using decision symbols in the next section.

Selection Statements

Python provides three types of selection statements that execute code based on a *condition*—an expression that evaluates to either `True` or `False`:

1. Bohm, C., and G. Jacopini, "Flow Diagrams, Turing Machines, and Languages with Only Two Formation Rules," *Communications of the ACM*, Vol. 9, No. 5, May 1966, pp. 336–371.

- The `if` statement performs an action if a condition is `True` or skips the action if the condition is `False`.
- The **`if...else` statement** performs an action if a condition is `True` or performs a *different* action if the condition is `False`.
- The **`if...elif...else` statement** performs one of many different actions, depending on the truth or falsity of *several* conditions.

Anywhere a single action can be placed, a group of actions can be placed.

The `if` statement is called a **single-selection statement** because it selects or ignores a *single* action (or group of actions). The `if...else` statement is called a **double-selection statement** because it selects between *two different* actions (or groups of actions). The `if...elif...else` statement is called a **multiple-selection statement** because it selects one of many different actions (or groups of actions).

Repetition Statements

Python provides two repetition statements—`while` and `for`:

- The `while` statement repeats an action (or a group of actions) as long as a condition remains `True`.
- The `for` statement repeats an action (or a group of actions) for every item in a sequence of items.

Keywords

The words `if`, `elif`, `else`, `while`, `for`, `True` and `False` are keywords that Python reserves to implement its features, such as control statements. Using a keyword as an identifier such as a variable name is a syntax error. The following table lists Python's keywords.

Python keywords

`and`	`as`	`assert`	`async`	`await`	`break`	`class`
`continue`	`def`	`del`	`elif`	`else`	`except`	`False`
`finally`	`for`	`from`	`global`	`if`	`import`	`in`
`is`	`lambda`	`None`	`nonlocal`	`not`	`or`	`pass`
`raise`	`return`	`True`	`try`	`while`	`with`	`yield`

Control Statements Summary

You form each Python program by combining as many control statements of each type as you need for the algorithm the program implements. With **Single-entry/single-exit** (one way in/one way out) **control statements**, the exit point of one connects to the entry point of the next. This is similar to the way a child stacks building blocks—hence, the term **control-statement stacking**. **Control-statement nesting** also connects control statements—we'll see how later in the chapter.

You can construct any Python program from only six different forms of control (sequential execution, and the `if`, `if...else`, `if...elif...else`, `while` and `for` statements). You combine these in only two ways (control-statement stacking and control-statement nesting). This is the essence of simplicity.

Self Check

1 *(Fill-In)* You can write all programs using three forms of control—_____, _____ and _____.
Answer: sequential execution, selection statements, repetition statements.

2 *(Fill-In)* A(n) _____ is a graphical representation of an algorithm.
Answer: flowchart.

3.5 `if` Statement

Suppose that a passing grade on an examination is 60. The pseudocode

```
If student's grade is greater than or equal to 60
    Display 'Passed'
```

determines whether the condition "student's grade is greater than or equal to 60" is true or false. If the condition is true, 'Passed' is displayed. Then, the next pseudocode statement in order is "performed." (Remember that pseudocode is not a real programming language.) If the condition is false, nothing is displayed, and the next pseudocode statement is "performed." The pseudocode's second line is indented. Python code requires indentation. Here it emphasizes that 'Passed' is displayed *only* if the condition is true.

Let's assign `85` to the variable `grade`, then show and execute the Python `if` statement for the pseudocode:

```
In [1]: grade = 85

In [2]: if grade >= 60:
   ...:     print('Passed')
   ...:
Passed
```

The `if` statement closely resembles the pseudocode. The condition `grade >= 60` is `True`, so the indented `print` statement displays `'Passed'`.

Suite Indentation

Indenting a suite is required; otherwise, an `IndentationError` syntax error occurs:

```
In [3]: if grade >= 60:
   ...: print('Passed')  # statement is not indented properly
  File "<ipython-input-3-f42783904220>", line 2
    print('Passed')  # statement is not indented properly
        ^
IndentationError: expected an indented block
```

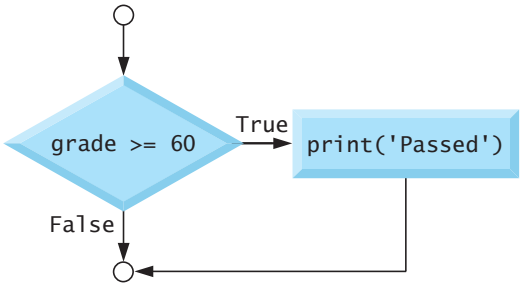
An `IndentationError` also occurs if you have more than one statement in a suite and those statements do not have the *same* indentation:

```
In [4]: if grade >= 60:
   ...:     print('Passed')  # indented 4 spaces
   ...:   print('Good job!')  # incorrectly indented only two spaces
  File <ipython-input-4-8c0d75c127bf>, line 3
    print('Good job!')  # incorrectly indented only two spaces
    ^
IndentationError: unindent does not match any outer indentation level
```

Sometimes error messages may not be clear. The fact that Python calls attention to the line is usually enough for you to figure out what's wrong. Apply indentation conventions uniformly throughout your code. Programs that are not uniformly indented are hard to read.

if Statement Flowchart

The flowchart for the `if` statement in snippet `[2]` is:



The decision (diamond) symbol contains a condition that can be either `True` or `False`. The diamond has two flowlines emerging from it:

- One indicates the direction to follow when the condition in the symbol is `True`. This points to the action (or group of actions) that should execute.
- The other indicates the direction to follow when the condition is `False`. This skips the action (or group of actions).

Every Expression Can Be Interpreted as Either True or False

You can base decisions on *any* expression. A nonzero value is `True`. Zero is `False`:

```
In [5]: if 1:
   ...:     print('Nonzero values are true, so this will print')
   ...:
Nonzero values are true, so this will print

In [6]: if 0:
   ...:     print('Zero is false, so this will not print')

In [7]:
```

Strings containing characters are `True` and empty strings (`''`, `""` or `"""""""`) are `False`.

An Additional Note on Confusing == and =

Using the equality operator `==` instead of the assignment symbol `=` in an assignment statement can lead to subtle problems. For example, in this session, snippet `[1]` defined `grade` with the assignment:

```
grade = 85
```

If instead we accidentally wrote:

```
grade == 85
```

then `grade` would be undefined and we'd get a `NameError`.

If `grade` had been defined before the preceding statement, then `grade == 85` would evaluate to `True` or `False`, depending on `grade`'s value, and not perform the intended assignment. This is a logic error.

Self Check

1 *(True/False)* If you indent a suite's statements, you will not get an `IndentationError`.
Answer: False. All the statements in a suite must have the *same* indentation. Otherwise, an `IndentationError` occurs.

2 *(IPython Session)* Redo this section's snippets `[1]` and `[2]`, then change `grade` to `55` and repeat the `if` statement to show that its suite does not execute. The next section shows how to recall and re-execute earlier snippets to avoid having to re-enter the code.
Answer:

```
In [1]: grade = 85

In [2]: if grade >= 60:
   ...:     print('Passed')
   ...:
Passed

In [3]: grade = 55

In [4]: if grade >= 60:
   ...:     print('Passed')
   ...:

In [5]:
```

3.6 `if...else` and `if...elif...else` Statements

The `if...else` statement performs different suites, based on whether a condition is `True` or `False`. The pseudocode below displays 'Passed' if the student's grade is greater than or equal to 60; otherwise, it displays 'Failed':

```
If student's grade is greater than or equal to 60
    Display 'Passed'
Else
    Display 'Failed'
```

In either case, the next pseudocode statement in sequence after the entire *If*...*Else* is "performed." We indent both the *If* and *Else* suites, and by the same amount. Let's create and **initialize** (that is, give a starting value to) the variable `grade`, then show and execute the Python `if...else` statement for the preceding pseudocode:

```
In [1]: grade = 85

In [2]: if grade >= 60:
   ...:     print('Passed')
   ...: else:
   ...:     print('Failed')
   ...:
Passed
```

The condition above is `True`, so the `if` suite displays `'Passed'`. Note that when you press *Enter* after typing `print('Passed')`, IPython indents the next line four spaces. You must delete those four spaces so that the `else:` suite correctly aligns under the `i` in `if`.

The following code assigns 57 to the variable grade, then shows the if...else statement again to demonstrate that only the else suite executes when the condition is False:

```
In [3]: grade = 57

In [4]: if grade >= 60:
   ...:     print('Passed')
   ...: else:
   ...:     print('Failed')
   ...:
Failed
```

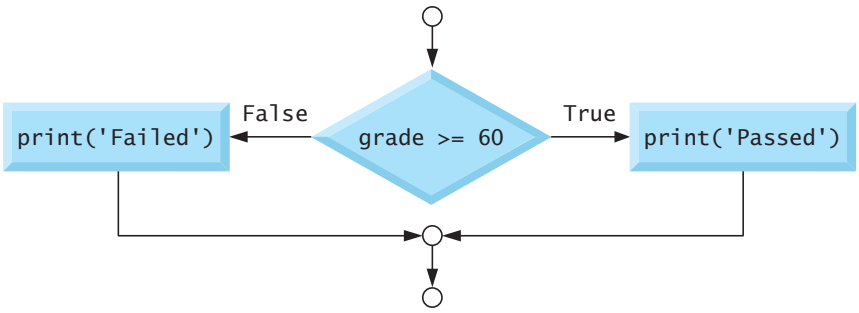
The up and down arrow keys navigate backwards and forwards through the current interactive session's snippets. Pressing *Enter* re-executes the snippet that's displayed. Let's set grade to 99, press the up arrow key twice to recall the code from snippet [4], then press *Enter* to re-execute that code as snippet [6]. Every recalled snippet that you execute gets a new ID:

```
In [5]: grade = 99

In [6]: if grade >= 60:
   ...:     print('Passed')
   ...: else:
   ...:     print('Failed')
   ...:
Passed
```

if...else Statement Flowchart

The flowchart below shows the preceding if...else statement's flow of control:



Conditional Expressions

Sometimes the suites in an if...else statement assign different values to a variable, based on a condition, as in:

```
In [7]: grade = 87

In [8]: if grade >= 60:
   ...:     result = 'Passed'
   ...: else:
   ...:     result = 'Failed'
   ...:
```

We can then `print` or evaluate that variable:

```
In [9]: result
Out[9]: 'Passed'
```

You can write statements like snippet `[8]` using a concise **conditional expression**:

```
In [10]: result = ('Passed' if grade >= 60 else 'Failed')

In [11]: result
Out[11]: 'Passed'
```

The parentheses are not required, but they make it clear that the statement assigns the conditional expression's value to `result`. First, Python evaluates the condition `grade >= 60`:

- If it's `True`, snippet `[10]` assigns to `result` the value of the expression to the *left* of `if`, namely `'Passed'`. The `else` part does not execute.
- If it's `False`, snippet `[10]` assigns to `result` the value of the expression to the *right* of `else`, namely `'Failed'`.

In interactive mode, you also can evaluate the conditional expression directly, as in:

```
In [12]: 'Passed' if grade >= 60 else 'Failed'
Out[12]: 'Passed'
```

Multiple Statements in a Suite

The following code shows two statements in the `else` suite of an `if...else` statement:

```
In [13]: grade = 49

In [14]: if grade >= 60:
    ...:     print('Passed')
    ...: else:
    ...:     print('Failed')
    ...:     print('You must take this course again')
    ...:
Failed
You must take this course again
```

In this case, `grade` is less than `60`, so *both* statements in the `else`'s suite execute. If you do not indent the second `print`, then it's not in the `else`'s suite. So, that statement *always* executes, creating strange incorrect output:

```
In [15]: grade = 100

In [16]: if grade >= 60:
    ...:     print('Passed')
    ...: else:
    ...:     print('Failed')
    ...: print('You must take this course again')
    ...:
Passed
You must take this course again
```

if...elif...else Statement

You can test for many cases using the **if...elif...else statement**. The following pseudocode displays "A" for grades greater than or equal to 90, "B" for grades in the range 80–89, "C" for grades 70–79, "D" for grades 60–69 and "F" for all other grades:

```
If student's grade is greater than or equal to 90
    Display "A"
Else If student's grade is greater than or equal to 80
    Display "B"
Else If student's grade is greater than or equal to 70
    Display "C"
Else If student's grade is greater than or equal to 60
    Display "D"
Else
    Display "F"
```

Only the action for the first `True` condition executes. Let's show and execute the Python code for the preceding pseudocode. The pseudocode *Else If* is written with the keyword `elif`. Snippet `[18]` displays `C`, because `grade` is `77`:

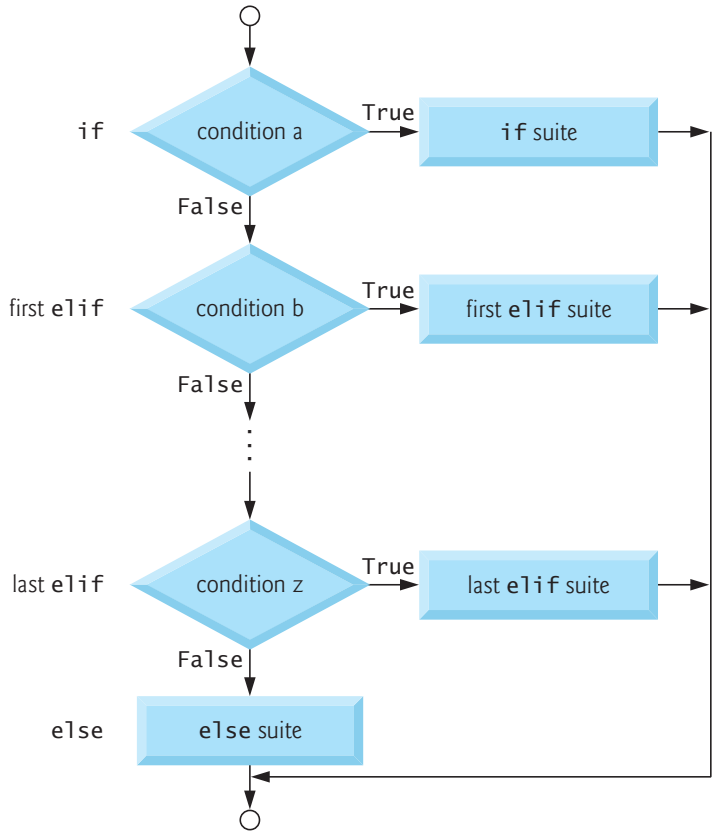
```
In [17]: grade = 77

In [18]: if grade >= 90:
    ...:     print('A')
    ...: elif grade >= 80:
    ...:     print('B')
    ...: elif grade >= 70:
    ...:     print('C')
    ...: elif grade >= 60:
    ...:     print('D')
    ...: else:
    ...:     print('F')
    ...:
C
```

The first condition—`grade >= 90`—is `False`, so `print('A')` is skipped. The second condition—`grade >= 80`—also is `False`, so `print('B')` is skipped. The third condition—`grade >= 70`—is `True`, so `print('C')` executes. Then all the remaining code in the `if...elif...else` statement is skipped. An `if...elif...else` is faster than separate `if` statements, because condition testing stops as soon as a condition is `True`.

if...elif...else Statement Flowchart

The following flowchart shows the general flow through an `if...elif...else` statement. It shows that, after any suite executes, control immediately exits the statement. The words to the left are not part of the flowchart. We added them to show how the flowchart corresponds to the equivalent Python code.



else Is Optional

The `else` in the `if...elif...else` statement is optional. Including it enables you to handle values that do not satisfy *any* of the conditions. When an `if...elif` statement without an `else` tests a value that does not make any of its conditions `True`, the program does not execute any of the statement's suites. The next statement in sequence after the `if...elif` statement executes. If you specify the `else`, you must place it after the last `elif`; otherwise, a `SyntaxError` occurs.

Logic Errors

The incorrectly indented code segment in snippet `[16]` is an example of a **nonfatal logic error**. The code executes, but it produces incorrect results. For a **fatal logic error** in a script, an exception occurs (such as a `ZeroDivisionError` from an attempt to divide by 0), so Python displays a traceback, then terminates the script. A fatal error in interactive mode terminates only the current snippet. Then IPython waits for your next input.

Self Check

1 *(True/False)* A fatal logic error causes a script to produce incorrect results, then continue executing.

Answer: False. A fatal logic error causes a script to terminate.

2 *(IPython Session)* Show that a `SyntaxError` occurs if an `if...elif` statement specifies an `else` before the last `elif`.

Answer:

```
In [1]: grade = 80

In [2]: if grade >= 90:
   ...:     print('A')
   ...: else:
   ...:     print('Not A or B')
   ...: elif grade >= 80:
  File "<ipython-input-2-033bcba40157>", line 5
    elif grade >= 80:
       ^
SyntaxError: invalid syntax
```

3.7 while Statement

The **while statement** allows you to *repeat* one or more actions while a condition remains `True`. Such a statement often is called a **loop**.

The following pseudocode specifies what happens when you go shopping:

```
While there are more items on my shopping list
    Buy next item and cross it off my list
```

If the condition "there are more items on my shopping list" is *true*, you perform the action "Buy next item and cross it off my list." You *repeat* this action while the condition remains *true*. You stop repeating this action when the condition becomes *false*—that is, when you've crossed all items off your shopping list.

Let's use a `while` statement to find the first power of 3 larger than 50:

```
In [1]: product = 3

In [2]: while product <= 50:
   ...:     product = product * 3
   ...:

In [3]: product
Out[3]: 81
```

First, we create `product` and initialize it to `3`. Then the `while` statement executes as follows:

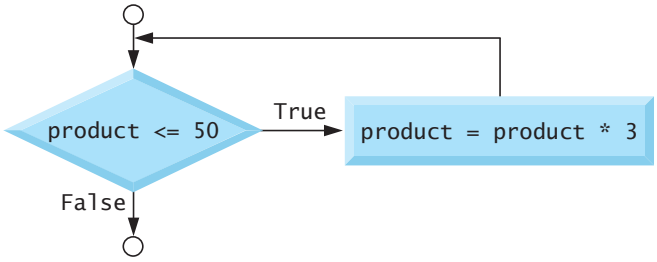
1. Python tests the condition `product <= 50`, which is `True` because `product` is `3`. The statement in the suite multiplies `product` by 3 and assigns the result (`9`) to `product`. One *iteration of the loop* is now complete.
2. Python again tests the condition, which is `True` because `product` is now `9`. The suite's statement sets `product` to `27`, completing the second iteration of the loop.
3. Python again tests the condition, which is `True` because `product` is now `27`. The suite's statement sets `product` to `81`, completing the third iteration of the loop.
4. Python again tests the condition, which is finally `False` because `product` is now `81`. The repetition now terminates.

Snippet `[3]` evaluates `product` to see its value, `81`, which is the first power of `3` larger than `50`. If this `while` statement were part of a larger script, execution would continue with the next statement in sequence after the `while`.

Something in the `while` statement's suite must change `product`'s value, so the condition eventually becomes `False`. Otherwise, a logic error called an **infinite loop** occurs. Such an error prevents the `while` statement from ever terminating—the program appears to "hang." In applications executed from a Terminal, Command Prompt or shell, type *Ctrl* + *c* or *control* + *c* (depending on your keyboard) to terminate an infinite loop. IDEs typically have a toolbar button or menu option for stopping a program's execution.

`while` Statement Flowchart

The following flowchart shows the preceding `while` statement's flow of control:



Follow the flowlines to experience the repetition. The flowline from the rectangle "closes the loop" by flowing back into the condition `product <= 50` that's tested during each iteration. When that condition becomes `False`, the `while` statement exits and control proceeds to the next statement in sequence.

Self Check

1 *(True/False)* A `while` statement performs its suite while some condition remains `True`.
Answer: True.

2 *(IPython Session)* Write statements to determine the first power of 7 greater than 1000.
Answer:

```
In [1]: product = 7

In [2]: while product <= 1000:
   ...:     product = product * 7
   ...:

In [3]: product
Out[3]: 2401
```

3.8 for Statement

Like the `while` statement, the **`for` statement** allows you to *repeat* an action or several actions. The `for` statement performs its action(s) for each item in a **sequence** of items. For example, a string is a sequence of individual characters. Let's display `'Programming'` with its characters separated by two spaces:

```
In [1]: for character in 'Programming':
   ...:     print(character, end='  ')
   ...:
P  r  o  g  r  a  m  m  i  n  g
```

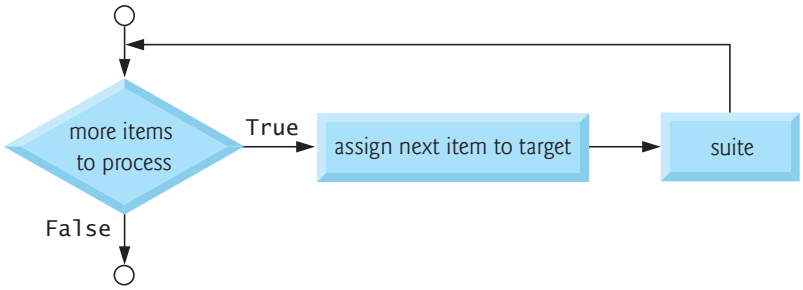
The for statement executes as follows:

- Upon entering the statement, it assigns the 'P' in 'Programming' to the **target** variable between keywords for and in—in this case, character.
- Next, the statement in the suite executes, displaying character's value followed by two spaces—we'll say more about this momentarily.
- After executing the suite, Python assigns to character the next item in the sequence (that is, the 'r' in 'Programming'), then executes the suite again.
- This continues while there are more items in the sequence to process. In this case, the statement terminates after displaying the letter 'g', followed by two spaces.

Using the target in the suite, as we did here to display its value, is common but not required.

for Statement Flowchart

The for statement's flowchart is similar to that of the while statement:



First, Python determines whether there are more items to process. If so, the for statement assigns the next item to the target, then performs the suite's action(s).

Function print's end Keyword Argument

The built-in function print displays its argument(s), then moves the cursor to the next line. You can change this behavior with the argument **end**, as in

```
print(character, end='  ')
```

We used two spaces (' '), so each call to print displays character's value followed by two spaces. So, all the characters display horizontally on the *same* line. Python calls end a **keyword argument**, but end is not a Python keyword. The end keyword argument is *optional*. If you do not include it, print uses a newline ('\n') by default. The *Style Guide for Python Code* recommends placing no spaces around a keyword argument's =. Keyword arguments are sometimes called named arguments.

Function print's sep Keyword Argument

You can use the keyword argument **sep** (short for separator) to specify the string that appears *between* the items that print displays. When you do not specify this argument, print uses a space character by default. Let's display three numbers, each separated from the next by a comma and a space, rather than just a space:

```
In [2]: print(10, 20, 30, sep=', ')
10, 20, 30
```

To remove the spaces, use an **empty string** with no characters between its quotes.

3.8.1 Iterables, Lists and Iterators

The sequence to the right of the `for` statement's `in` keyword must be an **iterable**. An iterable is an object from which the `for` statement can take one item at a time until no more items remain. Python has other iterable sequence types besides strings. One of the most common is a **list**, which is a comma-separated collection of items enclosed in square brackets (`[` and `]`). The following code totals five integers in a list:

```
In [3]: total = 0

In [4]: for number in [2, -3, 0, 17, 9]:
   ...:     total = total + number
   ...:

In [5]: total
Out[5]: 25
```

Each sequence has an **iterator**. The `for` statement uses the iterator "behind the scenes" to get each consecutive item until there are no more to process. The iterator is like a bookmark—it always knows where it is in the sequence, so it can return the next item when it's called upon to do so.

For each `number` in the list, the suite adds the `number` to the `total`. When there are no more items to process, `total` contains the sum (`25`) of the list's items. We cover lists in detail in the "Sequences: Lists and Tuples" chapter. There, you'll see that the order of the items in a list matters and that a list's items are **mutable** (that is, modifiable).

3.8.2 Built-In `range` Function

Let's use a `for` statement and the built-in **`range` function** to iterate precisely `10` times, displaying the values from `0` through `9`:

```
In [6]: for counter in range(10):
   ...:     print(counter, end=' ')
   ...:
0 1 2 3 4 5 6 7 8 9
```

The function call `range(10)` creates an iterable object that represents a sequence of consecutive integer values starting from `0` and continuing up to, but *not* including, the argument value (`10`). In this case, the sequence is `0`, `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`. The `for` statement exits when it finishes processing the last integer that `range` produces. Iterators and iterable objects are two of Python's *functional-style programming* features. We'll introduce more of these throughout the book.

Off-By-One Errors

A logic error known as an **off-by-one error** occurs when you assume that `range`'s argument value is included in the generated sequence. For example, if you provide `9` as `range`'s argument when trying to produce the sequence `0` through `9`, `range` generates only `0` through `8`.

Self Check

1 *(Fill-In)* Function _____ generates a sequence of integers.

Answer: `range`.

2 *(IPython Session)* Use the `range` function and a `for` statement to calculate the total of the integers from 0 through 1,000,000.
Answer:

```
In [1]: total = 0

In [2]: for number in range(1000001):
   ...:     total = total + number
   ...:

In [3]: total
Out[3]: 500000500000
```

3.9 Augmented Assignments

Augmented assignments abbreviate assignment expressions in which the same variable name appears on the left and right of the assignment's =, as `total` does in:

```
for number in [1, 2, 3, 4, 5]:
    total = total + number
```

Snippet [2] reimplements this using an **addition augmented assignment (+=) statement**:

```
In [1]: total = 0

In [2]: for number in [1, 2, 3, 4, 5]:
   ...:     total += number  # add number to total and store in number
   ...:

In [3]: total
Out[3]: 15
```

The += expression in snippet [2] first adds `number`'s value to the current `total`, then stores the new value in `total`. The table below shows sample augmented assignments:

Augmented assignment	Sample expression	Explanation	Assigns
Assume: `c = 3, d = 5, e = 4, f = 2, g = 9, h = 12`			
`+=`	`c += 7`	`c = c + 7`	10 to c
`-=`	`d -= 4`	`d = d - 4`	1 to d
`*=`	`e *= 5`	`e = e * 5`	20 to e
`**=`	`f **= 3`	`f = f ** 3`	8 to f
`/=`	`g /= 2`	`g = g / 2`	4.5 to g
`//=`	`g //= 2`	`g = g // 2`	4 to g
`%=`	`h %= 9`	`h = h % 9`	3 to h

Self Check

1 *(Fill-In)* If `x` is `7`, the value of `x` after evaluating `x *= 5` is _____.
Answer: `35`.

2 *(IPython Session)* Create a variable `x` with the value `12`. Use an exponentiation augmented assignment statement to square `x`'s value. Show `x`'s new value.
Answer:

```
In [1]: x = 12

In [2]: x **= 2

In [3]: x
Out[3]: 144
```

3.10 Program Development: Sequence-Controlled Repetition

Experience has shown that the most challenging part of solving a problem on a computer is developing an algorithm for the solution. As you'll see, once a correct algorithm has been specified, creating a working Python program from the algorithm is typically straightforward. This section and the next present problem solving and program development by creating scripts that solve two class-averaging problems.

3.10.1 Requirements Statement

A **requirements statement** describes *what* a program is supposed to do, but not *how* the program should do it. Consider the following simple requirements statement:

> *A class of ten students took a quiz. Their grades (integers in the range 0 – 100) are 98, 76, 71, 87, 83, 90, 57, 79, 82, 94. Determine the class average on the quiz.*

Once you know the problem's requirements, you can begin creating an algorithm to solve it. Then, you can implement that solution as a program.

The algorithm for solving this problem must:

1. Keep a running total of the grades.
2. Calculate the average—the total of the grades divided by the number of grades.
3. Display the result.

For this example, we'll place the 10 grades in a list. You also could input the grades from a user at the keyboard (as we'll do in the next example) or read them from a file (as you'll see how to do in the "Files and Exceptions" chapter). We also show you how to read data from SQL and NoSQL databases in later chapters.

3.10.2 Pseudocode for the Algorithm

The following pseudocode lists the actions to execute and specifies the order in which they should execute:

```
Set total to zero
Set grade counter to zero
Set grades to a list of the ten grades

For each grade in the grades list:
    Add the grade to the total
    Add one to the grade counter

Set the class average to the total divided by the number of grades
Display the class average
```

Note the mentions of *total* and *grade counter*. In Fig. 3.1's script, the variable `total` (line 5) stores the `grade` values' running total, and `grade_counter` (line 6) counts the number of grades we've processed. We'll use these to calculate the average. Variables for totaling and counting normally are initialized to zero before they're used, as we do in lines 5 and 6.

3.10.3 Coding the Algorithm in Python

The following script implements the pseudocode algorithm.

```
1  # fig03_01.py
2  """Class average program with sequence-controlled repetition."""
3
4  # initialization phase
5  total = 0  # sum of grades
6  grade_counter = 0
7  grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94]  # list of 10 grades
8
9  # processing phase
10 for grade in grades:
11     total += grade  # add current grade to the running total
12     grade_counter += 1  # indicate that one more grade was processed
13
14 # termination phase
15 average = total / grade_counter
16 print(f'Class average is {average}')
```

```
Class average is 81.7
```

Fig. 3.1 | Class average program with sequence-controlled repetition.

Execution Phases

We used blank lines and comments to break the script into three **execution phases**—initialization, processing and termination:

- The **initialization phase** creates the variables needed to process the grades and set these variables to appropriate initial values.
- The **processing phase** processes the grades, calculating the running total and counting the number of grades processed so far.
- The **termination phase** calculates and displays the class average.

Many scripts can be **decomposed** (that is, broken apart) into these three phases.

Initialization Phase

Lines 5–6 create the variables `total` and `grade_counter` and initialize each to `0`. Line 7

```
grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94]  # list of 10 grades
```

creates the variable `grades` and initializes it with a list of 10 integer grades.

Processing Phase

The `for` statement processes each `grade` in the list `grades`. Line 11 adds the current `grade` to the `total`. Then, line 12 adds `1` to the variable `grade_counter` to keep track of the number of grades processed so far. Repetition terminates when all 10 grades in the list have been processed. This is called **definite repetition** because the number of repetitions is known before the loop begins executing. In this case, it's the number of elements in the list `grades`. The *Style Guide for Python Code* recommends placing a blank line above and below each control statement (as in lines 8 and 13).

Termination Phase

When the `for` statement terminates, line 15 calculates the average and assigns it to the variable `average`. Then line 16 displays `average`. Later in this chapter, we use functional-style programming features to calculate the average of a list's items more concisely.

3.10.4 Introduction to Formatted Strings

Line 16 uses the following simple **f-string** (short for **formatted string**) to format this script's result by inserting the value of `average` into a string:

```
f'Class average is {average}'
```

The letter `f` before the string's opening quote indicates it's an f-string. You specify where to insert values by using placeholders delimited by curly braces (`{` and `}`). The placeholder

```
{average}
```

converts the variable `average`'s value to a string representation, then replaces `{average}` with that **replacement text**. Replacement-text expressions may contain values, variables or other expressions, such as calculations or function calls. In line 16, we could have used `total / grade_counter` in place of `average`, eliminating the need for line 15.

Self Check

1 *(Fill-In)* A(n) _____ describes *what* a program is supposed to do, but not *how* the program should do it.
Answer: requirements statement.

2 *(Fill-In)* Many of the scripts you'll write can be decomposed into three phases: _____, _____ and _____.
Answer: initialization, processing, termination.

3 *(IPython Session)* Display an f-string in which you insert the values of the variables `number1` (`7`) and `number2` (`5`) and their product. The displayed string should be

```
7 times 5 is 35
```

Answer:

```
In [1]: number1 = 7

In [2]: number2 = 5

In [3]: print(f'{number1} times {number2} is {number1 * number2}')
7 times 5 is 35
```

3.11 Program Development: Sentinel-Controlled Repetition

Let's generalize the class-average problem. Consider the following requirements statement:

> *Develop a class-averaging program that processes an arbitrary number of grades each time the program executes.*

In the first class-average example, we knew in advance the 10 grades to process. The requirements statement does not state what the grades are or how many there are, so we're going to have the user enter the grades into the program. The program processes an arbitrary number of grades. How can the program determine when to stop processing grades so that it can move on to calculate and display the class average?

One way to solve this problem is to use a special value called a **sentinel value** (also called a **signal value**, a **dummy value** or a **flag value**) to indicate "end of data entry." This is a bit like the way a caboose "marks" the end of a train. The user enters grades one at a time until all the grades have been entered. The user then enters the sentinel value to indicate that there are no more grades. **Sentinel-controlled repetition** is often called **indefinite repetition** because the number of repetitions is *not* known before the loop begins executing.

A sentinel value must not be confused with any acceptable input value. Grades on a quiz are typically nonnegative integers between 0 and 100, so the value –1 is an acceptable sentinel value for this problem. Thus, a run of the class-average program might process a stream of inputs such as 95, 96, 75, 74, 89 and –1. The program would then compute and print the class average for the grades 95, 96, 75, 74 and 89. The sentinel value –1 should *not* enter into the averaging calculation.

Developing the Pseudocode Algorithm with Top-Down, Stepwise Refinement

We approach this class-average problem with a technique called **top-down, stepwise refinement**. We begin with a pseudocode representation of the top:

> *Determine the class average for the quiz*

The top is a single statement that conveys the program's overall function. Although it's a *complete* representation of a program, the top rarely conveys enough detail from which to write a program. The *top* specifies *what* should be done, but not *how* to implement it. So we begin the **refinement process**. We decompose the top into a sequence of smaller tasks—a process sometimes called **divide and conquer**. This results in the following **first refinement**:

> *Initialize variables*
> *Input, sum and count the quiz grades*
> *Calculate and display the class average*

Each refinement represents the *complete* algorithm—only the level of detail varies. In this refinement, the three pseudocode statements happen to correspond to the three execution phases described in the preceding section. The algorithm does not yet provide enough detail for us to write the Python program. So, we continue with the next refinement.

Second Refinement

To proceed to the **second refinement**, we commit to specific variables. The program needs to maintain

- a grade variable in which each successive user input will be stored,
- a running total of the grades,
- a count of how many grades have been processed and
- a variable that contains the calculated average.

The pseudocode statement

```
Initialize variables
```

can be refined as follows:

```
Initialize total to zero
Initialize grade counter to zero
```

Only the variables *total* and *grade counter* need to be initialized before they're used. We do not initialize the variables for the user input and calculated average. Their values will be replaced each time we input a grade from the user and when we calculate the class average, respectively. We'll create these variables when they're needed.

The next pseudocode statement requires a loop that successively inputs each grade:

```
Input, sum and count the quiz grades
```

We do not know how many grades will be entered, so we use sentinel-controlled repetition. The user enters legitimate grades successively. After the last legitimate grade has been entered, the user enters the sentinel value. The program tests for the sentinel value after each grade is input and terminates the loop when the sentinel has been entered. The second refinement of the preceding pseudocode statement is

```
Input the first grade (possibly the sentinel)
While the user has not entered the sentinel
    Add this grade into the running total
    Add one to the grade counter
    Input the next grade (possibly the sentinel)
```

The pseudocode statement

```
Calculate and display the class average
```

can be refined as follows:

```
If the counter is not equal to zero
    Set the average to the total divided by the grade counter
    Display the average
Else
    Display "No grades were entered"
```

Notice that we're testing for the possibility of division by zero. If undetected, this would cause a fatal logic error. In the "Files and Exceptions" chapter, we discuss how to write programs that recognize such exceptions and take appropriate actions.

The following is the class-average problem's complete second refinement:

```
Initialize total to zero
Initialize grade counter to zero

Input the first grade (possibly the sentinel)
While the user has not entered the sentinel
    Add this grade into the running total
    Add one to the grade counter
    Input the next grade (possibly the sentinel)

If the counter is not equal to zero
    Set the average to the total divided by the counter
    Display the average
Else
    Display "No grades were entered"
```

Sometimes more than two refinements are necessary. You stop refining when there is enough detail for you to convert the pseudocode to Python. We include blank lines for readability. Here, they happen to separate the algorithm into the three popular execution phases.

Implementing Sentinel-Controlled Iteration

The following script implements the pseudocode algorithm and shows a sample execution in which the user enters three grades and the sentinel value.

```
# fig03_02.py
"""Class average program with sentinel-controlled iteration."""

# initialization phase
total = 0  # sum of grades
grade_counter = 0  # number of grades entered

# processing phase
grade = int(input('Enter grade, -1 to end: '))  # get one grade

while grade != -1:
    total += grade
    grade_counter += 1
    grade = int(input('Enter grade, -1 to end: '))

# termination phase
if grade_counter != 0:
    average = total / grade_counter
    print(f'Class average is {average:.2f}')
else:
    print('No grades were entered')
```

Fig. 3.2 | Class average program with sentinel-controlled iteration. (Part 1 of 2.)

```
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 72
Enter grade, -1 to end: -1
Class average is 85.67
```

Fig. 3.2 | Class average program with sentinel-controlled iteration. (Part 2 of 2.)

Program Logic for Sentinel-Controlled Repetition

In sentinel-controlled repetition, the program reads the first value (line 9) before reaching the `while` statement. Line 9 demonstrates why we did not create the variable `grade` until we needed it in the program. If we had initialized it, that value would have been replaced immediately by this assignment.

The value input in line 9 determines whether the program's flow of control should enter the `while`'s suite (lines 12–14). If the condition in line 11 is `False`, the user entered the sentinel value (`-1`), so the suite does not execute because the user did not enter any grades. If the condition is `True`, the suite executes, adding the `grade` value to the `total` and incrementing the `grade_counter`. Next, line 14 inputs another grade from the user. Then, the `while`'s condition (line 11) is tested again, using the most recent `grade` entered by the user. The value of `grade` is always input immediately before the program tests the `while` condition, so we can determine whether the value just input is the sentinel before processing that value as a grade. When the sentinel value is input, the loop terminates, and the program does not add –1 to the total. In a sentinel-controlled loop that performs user input, any prompts (lines 9 and 14) should remind the user of the sentinel value.

After the loop terminates, the `if...else` statement (lines 17–21) executes. Line 17 determines whether the user entered any grades. If not, the `else` part (lines 20–21) executes and displays the message `'No grades were entered'` and the program terminates.

Formatting the Class Average with Two Decimal Places

This example formatted the class average with two digits to the right of the decimal point. In an f-string, you can optionally follow a replacement-text expression with a colon (`:`) and a **format specifier** that describes how to format the replacement text. The format specifier `.2f` (line 19) formats the `average` as a floating-point number (`f`) with two digits to the right of the decimal point (`.2`). In this example, the sum of the grades was 257, which, when divided by 3, yields 85.666666666.... Formatting the `average` with `.2f` *rounds* it to the hundredths position, producing the replacement text `85.67`. An average with only one digit to the right of the decimal point would be formatted with a **trailing zero** (e.g., `85.50`). The chapter "Strings: A Deeper Look" discusses many string-formatting features.

Control-Statement Stacking

In this example, notice that control statements are stacked in sequence. The `while` statement (lines 11–14) is followed immediately by an `if...else` statement (lines 17–21).

Self Check

1 *(Fill-In)* Sentinel-controlled repetition is called _____ because the number of repetitions is not known before the loop begins executing.

Answer: indefinite repetition.

2 ***(True/False)*** Sentinel-controlled repetition uses a counter variable to control the number of times a set of instructions executes.
Answer: False. Sentinel-control repetition terminates repetition when the sentinel value is encountered.

3.12 Program Development: Nested Control Statements

Let's work through another complete problem. Once again, we plan the algorithm using pseudocode and top-down, stepwise refinement and we develop a corresponding Python script. Consider the following requirements statement:

> *A college offers a course that prepares students for the state licensing exam for real-estate brokers. Last year, several of the students who completed this course took the licensing examination. The college wants to know how well its students did on the exam. You have been asked to write a program to summarize the results. You have been given a list of these 10 students. Next to each name is written a 1 if the student passed the exam and a 2 if the student failed.*
>
> *Your program should analyze the results of the exam as follows:*
>
> 1. *Input each test result (i.e., a 1 or a 2). Display the message "Enter result" each time the program requests another test result.*
> 2. *Count the number of test results of each type.*
> 3. *Display a summary of the test results indicating the number of students who passed and the number of students who failed.*
> 4. *If more than eight students passed the exam, display "Bonus to instructor."*

After reading the requirements statement carefully, we make the following observations about the problem:

1. The program must process 10 test results. We'll use a `for` statement and the `range` function to control repetition.
2. Each test result is a number—either a 1 or a 2. Each time the program reads a test result, the program must determine if the number is a 1 or a 2. We test for a 1 in our algorithm. If the number is not a 1, we assume that it's a 2. (An exercise at the end of the chapter considers the consequences of this assumption.)
3. We'll use two counters—one to count the number of students who passed the exam and one to count the number of students who failed.
4. After the script processes all the results, it must decide if more than eight students passed the exam so that it can bonus the instructor.

Top-Down, Stepwise Refinement

We begin with a pseudocode representation of the top:

Analyze exam results and decide whether instructor should receive a bonus

Once again, the top is a *complete* representation of the program, but several refinements are likely to be needed before the pseudocode can evolve naturally into a Python program.

First Refinement

Our first refinement is

```
Initialize variables
Input the ten exam grades and count passes and failures
Summarize the exam results and decide whether instructor should receive a bonus
```

Here, too, even though we have a *complete* representation of the entire program, further refinement is necessary. Note again that this first refinement happens to correspond to the three-execution-phases model.

Second Refinement

We now commit to specific variables. We need counters to record the passes and failures, and a variable to store the user input. The pseudocode statement

```
Initialize variables
```

can be refined as follows:

```
Initialize passes to zero
Initialize failures to zero
```

Only the counters for the number of passes and number of failures need to be initialized.

The pseudocode statement

```
Input the ten exam grades and count passes and failures
```

requires a loop that successively inputs the result of each exam. Here it's known in advance that there are ten exam results, so the `for` statement and the `range` function are appropriate. Inside the loop (that is, *nested* within the loop), an `if...else` statement determines whether each exam result is a pass or a failure and increments the appropriate counter. The refinement of the preceding pseudocode statement is

```
For each of the ten students
    Input the next exam result

    If the student passed
        Add one to passes
    Else
        Add one to failures
```

The blank line before the *If…Else* improves readability.

The pseudocode statement

```
Summarize the exam results and decide whether instructor should receive a bonus
```

may be refined as follows:

```
Display the number of passes
Display the number of failures

If more than eight students passed
    Display "Bonus to instructor"
```

Complete Pseudocode Algorithm

The pseudocode is now sufficiently refined for conversion to Python—the complete second refinement is shown below:

```
Initialize passes to zero
Initialize failures to zero

For each of the ten students
    Input the next exam result

    If the student passed
        Add one to passes
    Else
        Add one to failures

Display the number of passes
Display the number of failures

If more than eight students passed
    Display "Bonus to instructor"
```

Implementing the Algorithm

The following script implements the algorithm and is followed by two sample executions. Once again, notice that the Python code closely resembles the pseudocode. Lines 9–16 loop 10 times, inputting and processing one exam result each time. The `if...else` statement (lines 13–16) that processes each result is *nested* in the `for` statement—that is, it's part of the `for` statement's suite. If the `result` is `1`, we add `1` to `passes`; otherwise, we assume the `result` is `2` and add `1` to `failures`. After inputting 10 values, the loop terminates and lines 19 and 20 display `passes` and `failures`. Lines 22–23 determine whether more than eight students passed the exam and, if so, display `'Bonus to instructor'`.

```
1  # fig03_03.py
2  """Using nested control statements to analyze examination results."""
3
4  # initialize variables
5  passes = 0  # number of passes
6  failures = 0  # number of failures
7
8  # process 10 students
9  for student in range(10):
10     # get one exam result
11     result = int(input('Enter result (1=pass, 2=fail): '))
12
13     if result == 1:
14         passes = passes + 1
15     else:
16         failures = failures + 1
17
```

Fig. 3.3 | Analysis of examination results. (Part 1 of 2.)

```
# termination phase
print('Passed:', passes)
print('Failed:', failures)

if passes > 8:
    print('Bonus to instructor')
```

```
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Passed: 6
Failed: 4
```

```
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Passed: 9
Failed: 1
Bonus to instructor
```

Fig. 3.3 | Analysis of examination results. (Part 2 of 2.)

Self Check

1 *(IPython Session)* Use a `for` statement to input two integers. Use a nested `if...else` statement to display whether each value is even or odd. Enter `10` and `7` to test your code.

Answer:

```
In [1]: for count in range(2):
   ...:     value = int(input('Enter an integer: '))
   ...:     if value % 2 == 0:
   ...:         print(f'{value} is even')
   ...:     else:
   ...:         print(f'{value} is odd')
   ...:
Enter an integer: 10
10 is even
Enter an integer: 7
7 is odd
```

3.13 Built-In Function range: A Deeper Look

Function `range` also has two- and three-argument versions. As you've seen, `range`'s one-argument version produces a sequence of consecutive integers from `0` up to, but not including, the argument's value. Function `range`'s two-argument version produces a sequence of consecutive integers from its first argument's value up to, but not including, the second argument's value, as in:

```
In [1]: for number in range(5, 10):
   ...:     print(number, end=' ')
   ...:
5 6 7 8 9
```

Function `range`'s three-argument version produces a sequence of integers from its first argument's value up to, but not including, the second argument's value, *incrementing* by the third argument's value, which is known as the **step**:

```
In [2]: for number in range(0, 10, 2):
   ...:     print(number, end=' ')
   ...:
0 2 4 6 8
```

If the third argument is negative, the sequence progresses from the first argument's value *down* to, but not including the second argument's value, *decrementing* by the third argument's value, as in:

```
In [3]: for number in range(10, 0, -2):
   ...:     print(number, end=' ')
   ...:
10 8 6 4 2
```

Self Check

1 *(True/False)* Function call `range(1, 10)` generates the sequence 1 through 10.
Answer: False. Function call `range(1, 10)` generates the sequence 1 through 9.

2 *(IPython Session)* What happens if you try to print the items in `range(10, 0, 2)`?
Answer: Nothing displays because the step is not negative (this is not a fatal error):

```
In [1]: for number in range(10, 0, 2):
   ...:     print(number, end=' ')
   ...:

In [2]:
```

3 *(IPython Session)* Use a `for` statement, `range` and `print` to display on one line the sequence of values `99 88 77 66 55 44 33 22 11 0`, each separated by one space.
Answer:

```
In [3]: for number in range(99, -1, -11):
   ...:     print(number, end=' ')
   ...:
99 88 77 66 55 44 33 22 11 0
```

4 *(IPython Session)* Use `for` and `range` to sum the even integers from `2` through `100`, then display the sum.

Answer:

```
In [4]: total = 0

In [5]: for number in range(2, 101, 2):
   ...:     total += number
   ...:

In [6]: total
Out[6]: 2550
```

3.14 Using Type `Decimal` for Monetary Amounts

In this section, we introduce `Decimal` capabilities for precise monetary calculations. If you enter banking or other fields that require the accuracy provided by type `Decimal`, you should investigate `Decimal`'s capabilities in depth.

For most scientific and other mathematical applications that use numbers with decimal points, Python's built-in floating-point numbers work well. For example, when we speak of a "normal" body temperature of 98.6, we do not need to be precise to a large number of digits. When we view the temperature on a thermometer and read it as 98.6, the actual value may be 98.5999473210643. The point here is that calling this number 98.6 is adequate for most body-temperature applications.

Floating-point values are stored in binary format (we introduced binary in the first chapter and discuss it in depth in the online "Number Systems" appendix). Some floating-point values are represented only approximately when they're converted to binary. For example, consider the variable `amount` with the dollars-and-cents value `112.31`. If you display `amount`, it appears to have the exact value you assigned to it:

```
In [1]: amount = 112.31

In [2]: print(amount)
112.31
```

However, if you print `amount` with `20` digits of precision to the right of the decimal point, you can see that the actual floating-point value in memory is not exactly `112.31`—it's only an approximation:

```
In [3]: print(f'{amount:.20f}')
112.31000000000000227374
```

Many applications require *precise* representation of numbers with decimal points. Institutions like banks that deal with millions or even billions of transactions per day have to tie out their transactions "to the penny." Floating-point numbers can represent some but not all monetary amounts with to-the-penny precision.

The **Python Standard Library**[2] provides many predefined capabilities you can use in your Python code to avoid "reinventing the wheel." For monetary calculations and other applications that require precise representation and manipulation of numbers with decimal points, the Python Standard Library provides type **`Decimal`**, which uses a special coding scheme to solve the problem of to-the-penny precision. That scheme requires additional memory to hold the numbers and additional processing time to perform calcu-

2. `https://docs.python.org/3.7/library/index.html`.

lations but provides the to-the-penny precision required for monetary calculations. Banks also have to deal with other issues such as using a fair rounding algorithm when they're calculating daily interest on accounts. Type `Decimal` offers such capabilities.[3]

Importing Type `Decimal` from the `decimal` Module

We've used several built-in types—**`int`** (for integers, like `10`), **`float`** (for floating-point numbers, like `7.5`) and **`str`** (for strings like `'Python'`). The `Decimal` type is not built into Python. Rather, it's part of the Python Standard Library, which is divided into **modules**—groups of related capabilities. The **`decimal`** module defines type `Decimal` and its capabilities.

To use capabilities from a module, you must first **`import`** the entire module, as in

```
import decimal
```

and refer to the `Decimal` type as `decimal.Decimal`, or you must indicate a specific capability to import using **`from...import`**, as we do here:

```
In [4]: from decimal import Decimal
```

This imports only the type `Decimal` from the `decimal` module so that you can use it in your code. We'll discuss other `import` forms beginning in the next chapter.

Creating `Decimal`s

You typically create a `Decimal` from a string:

```
In [5]: principal = Decimal('1000.00')

In [6]: principal
Out[6]: Decimal('1000.00')

In [7]: rate = Decimal('0.05')

In [8]: rate
Out[8]: Decimal('0.05')
```

We'll soon use the variables `principal` and `rate` in a compound-interest calculation.

`Decimal` Arithmetic

`Decimal`s support the standard arithmetic operators `+`, `-`, `*`, `/`, `//`, `**` and `%`, as well as the corresponding augmented assignments:

```
In [9]: x = Decimal('10.5')

In [10]: y = Decimal('2')

In [11]: x + y
Out[11]: Decimal('12.5')

In [12]: x // y
Out[12]: Decimal('5')

In [13]: x += y

In [14]: x
Out[14]: Decimal('12.5')
```

3. For more `decimal` module features, visit `https://docs.python.org/3.7/library/decimal.html`.

You may perform arithmetic between `Decimal`s and integers, but not between `Decimal`s and floating-point numbers.

Compound-Interest Problem Requirements Statement

Let's compute compound interest using the `Decimal` type for *precise* monetary calculations. Consider the following requirements statement:

> *A person invests $1000 in a savings account yielding 5% interest. Assuming that the person leaves all interest on deposit in the account, calculate and display the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:*
>
> $a = p(1 + r)^n$
>
> *where*
>
> p is the original amount invested (i.e., the principal),
> r is the annual interest rate,
> n is the number of years and
> a is the amount on deposit at the end of the nth year.

Calculating Compound Interest

To solve this problem, let's use variables `principal` and `rate` that we defined in snippets `[5]` and `[7]`, and a `for` statement that performs the interest calculation for each of the 10 years the money remains on deposit. For each `year`, the loop displays a formatted string containing the year number and the `amount` on deposit at the end of that year:

```
In [15]: for year in range(1, 11):
    ...:     amount = principal * (1 + rate) ** year
    ...:     print(f'{year:>2}{amount:>10.2f}')
    ...:
 1   1050.00
 2   1102.50
 3   1157.62
 4   1215.51
 5   1276.28
 6   1340.10
 7   1407.10
 8   1477.46
 9   1551.33
10   1628.89
```

The algebraic expression $(1 + r)^n$ from the requirements statement is written as

```
(1 + rate) ** year
```

where variable `rate` represents r and variable `year` represents n.

Formatting the Year and Amount on Deposit

The statement

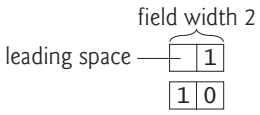
```
print(f'{year:>2}{amount:>10.2f}')
```

uses an f-string with two placeholders to format the loop's output.

The placeholder

```
{year:>2}
```

uses the format specifier `>2` to indicate that `year`'s value should be **right aligned** (`>`) in a field of width `2`—the **field width** specifies the number of character positions to use when displaying the value. For the single-digit `year` values `1`–`9`, the format specifier `>2` displays a space character followed by the value, thus right aligning the `year`s in the first column. The following diagram shows the numbers 1 and 10 each formatted in a field width of 2:

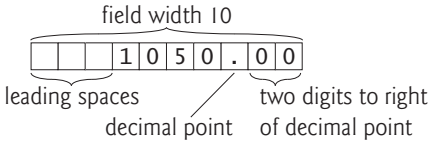


You can **left align** values with `<`.

The format specifier `10.2f` in the placeholder

```
{amount:>10.2f}
```

formats `amount` as a floating-point number (`f`) right aligned (`>`) in a field width of `10` with a decimal point and two digits to the right of the decimal point (`.2`). Formatting the `amount`s this way *aligns their decimal points vertically*, as is typical with monetary amounts. In the 10 character positions, the three rightmost characters are the number's decimal point followed by the two digits to its right. The remaining seven character positions are the leading spaces and the digits to the decimal point's left. In this example, all the dollar amounts have four digits to the left of the decimal point, so each number is formatted with three leading spaces. The following diagram shows the formatting for the value `1050.00`:



Self Check

1 *(Fill-In)* A field width specifies the _____ to use when displaying a value.

Answer: number of character positions.

2 *(IPython Session)* Assume that the tax on a restaurant bill is 6.25% and that the bill amount is $37.45. Use type `Decimal` to calculate the bill total, then print the result with two digits to the right of the decimal point.

Answer:

```
In [1]: from decimal import Decimal

In [2]: print(f"{Decimal('37.45') * Decimal('1.0625'):.2f}")
39.79
```

3.15 break and continue Statements

The `break` and `continue` statements alter a loop's flow of control. Executing a **`break`** statement in a `while` or `for` immediately exits that statement. In the following code, `range` produces the integer sequence 0–99, but the loop terminates when `number` is `10`:

```
In [1]: for number in range(100):
   ...:     if number == 10:
   ...:         break
   ...:     print(number, end=' ')
   ...:
0 1 2 3 4 5 6 7 8 9
```

In a script, execution would continue with the next statement after the `for` loop. The `while` and `for` statements each have an optional `else` clause that executes only if the loop terminates normally—that is, not as a result of a `break`. We explore this in the exercises.

Executing a **`continue`** statement in a `while` or `for` loop skips the remainder of the loop's suite. In a `while`, the condition is then tested to determine whether the loop should continue executing. In a `for`, the loop processes the next item in the sequence (if any):

```
In [2]: for number in range(10):
   ...:     if number == 5:
   ...:         continue
   ...:     print(number, end=' ')
   ...:
0 1 2 3 4 6 7 8 9
```

3.16 Boolean Operators and, or and not

The conditional operators >, <, >=, <=, == and != can be used to form simple conditions such as `grade >= 60`. To form more complex conditions that combine simple conditions, use the `and`, `or` and `not` Boolean operators.

Boolean Operator and

To ensure that two conditions are *both* `True` before executing a control statement's suite, use the **Boolean and operator** to combine the conditions. The following code defines two variables, then tests a condition that's `True` if and only if *both* simple conditions are `True`—if either (or both) of the simple conditions is `False`, the entire `and` expression is `False`:

```
In [1]: gender = 'Female'

In [2]: age = 70

In [3]: if gender == 'Female' and age >= 65:
   ...:     print('Senior female')
   ...:
Senior female
```

The `if` statement has two simple conditions:

- `gender == 'Female'` determines whether a person is a female and
- `age >= 65` determines whether that person is a senior citizen.

The simple condition to the left of the `and` operator evaluates first because == has higher precedence than `and`. If necessary, the simple condition to the right of `and` evaluates next, because >= has higher precedence than `and`. (We'll discuss shortly why the right side of an `and` operator evaluates *only* if the left side is `True`.) The entire `if` statement condition is `True` if and only if *both* of the simple conditions are `True`. The combined condition can be made clearer by adding redundant (unnecessary) parentheses

```
(gender == 'Female') and (age >= 65)
```

The table below summarizes the `and` operator by showing all four possible combinations of `False` and `True` values for *expression1* and *expression2*—such tables are called *truth tables*:

expression1	*expression2*	*expression1* `and` *expression2*
`False`	`False`	`False`
`False`	`True`	`False`
`True`	`False`	`False`
`True`	`True`	`True`

Boolean Operator `or`

Use the **Boolean `or` operator** to test whether one *or* both of two conditions are `True`. The following code tests a condition that's `True` if either *or* both simple conditions are `True`—the entire condition is `False` only if *both* simple conditions are `False`:

```
In [4]: semester_average = 83

In [5]: final_exam = 95

In [6]: if semester_average >= 90 or final_exam >= 90:
   ...:     print('Student gets an A')
   ...:
Student gets an A
```

Snippet `[6]` also contains two simple conditions:

- `semester_average >= 90` determines whether a student's average was an `A` (90 or above) during the semester, and
- `final_exam >= 90` determines whether a student's final-exam grade was an `A`.

The truth table below summarizes the Boolean `or` operator. Operator `and` has higher precedence than `or`.

expression1	*expression2*	*expression1* `or` *expression2*
`False`	`False`	`False`
`False`	`True`	`True`
`True`	`False`	`True`
`True`	`True`	`True`

Improving Performance with Short-Circuit Evaluation

Python stops evaluating an `and` expression as soon as it knows whether the entire condition is `False`. Similarly, Python stops evaluating an `or` expression as soon as it knows whether the entire condition is `True`. This is called *short-circuit evaluation*. So the condition

```
gender == 'Female' and age >= 65
```

stops evaluating immediately if `gender` is not equal to `'Female'` because the entire expression must be `False`. If `gender` is equal to `'Female'`, execution continues, because the entire expression will be `True` if the `age` is greater than or equal to `65`.

Similarly, the condition

```
semester_average >= 90 or final_exam >= 90
```

stops evaluating immediately if `semester_average` is greater than or equal to `90` because the entire expression must be `True`. If `semester_average` is less than `90`, execution continues, because the expression could still be `True` if the `final_exam` is greater than or equal to `90`.

In operator expressions that use `and`, make the condition that's more likely to be `False` the leftmost condition. In `or` operator expressions, make the condition that's more likely to be `True` the leftmost condition. These can reduce a program's execution time.

Boolean Operator `not`

The **Boolean `not` operator** "reverses" the meaning of a condition—`True` becomes `False` and `False` becomes `True`. This is a **unary operator**—it has only *one* operand. You place the `not` operator before a condition to choose a path of execution if the original condition (without the `not` operator) is `False`, such as in the following code:

```
In [7]: grade = 87

In [8]: if not grade == -1:
   ...:     print('The next grade is', grade)
   ...:
The next grade is 87
```

Often, you can avoid using `not` by expressing the condition in a more "natural" or convenient manner. For example, the preceding `if` statement can also be written as follows:

```
In [9]: if grade != -1:
   ...:     print('The next grade is', grade)
   ...:
The next grade is 87
```

The truth table below summarizes the `not` operator.

expression	`not` expression
`False`	`True`
`True`	`False`

The following table shows the precedence and grouping of the operators introduced so far, from top to bottom, in decreasing order of precedence.

Operators	Grouping
`()`	left to right
`**`	right to left
`*  /  //  %`	left to right
`+  -`	left to right
`<  <=  >  >=  ==  !=`	left to right
`not`	left to right
`and`	left to right
`or`	left to right

Self Check

1 *(IPython Session)* Assume that `i = 1`, `j = 2`, `k = 3` and `m = 2`. What does each of the following conditions display?

a) `(i >= 1) and (j < 4)`
b) `(m <= 99) and (k < m)`
c) `(j >= i) or (k == m)`
d) `(k + m < j) or (3 - j >= k)`
e) `not (k > m)`

Answer:

```
In [1]: i = 1

In [2]: j = 2

In [3]: k = 3

In [4]: m = 2

In [5]: (i >= 1) and (j < 4)
Out[5]: True

In [6]: (m <= 99) and (k < m)
Out[6]: False

In [7]: (j >= i) or (k == m)
Out[7]: True

In [8]: (k + m < j) or (3 - j >= k)
Out[8]: False

In [9]: not (k > m)
Out[9]: False
```

3.17 Intro to Data Science: Measures of Central Tendency—Mean, Median and Mode

Here we continue our discussion of using statistics to analyze data with several additional descriptive statistics, including:

- **mean**—the average value in a set of values.
- **median**—the middle value when all the values are arranged in sorted order.
- **mode**—the most frequently occurring value.

These are **measures of central tendency**—each is a way of producing a single value that represents a "central" value in a set of values, i.e., a value which is in some sense typical of the others.

Let's calculate the mean, median and mode on a list of integers. The following session creates a list called `grades`, then uses the built-in **`sum`** and **`len`** functions to calculate the mean "by hand"—`sum` calculates the total of the grades (397) and `len` returns the number of grades (5):

```
In [1]: grades = [85, 93, 45, 89, 85]

In [2]: sum(grades) / len(grades)
Out[2]: 79.4
```

The previous chapter mentioned the descriptive statistics count and sum—implemented in Python as the built-in functions `len` and `sum`. Like functions `min` and `max` (introduced in the preceding chapter), `sum` and `len` are both examples of functional-style programming *reductions*—they reduce a collection of values to a single value—the sum of those values and the number of values, respectively. In Fig. 3.1's class-average example, we could have deleted lines 10–15 and replaced `average` in line 16 with snippet `[2]`'s calculation.

The Python Standard Library's **`statistics` module** provides functions for calculating the mean, median and mode—these, too, are reductions. To use these capabilities, first `import` the `statistics` module:

```
In [3]: import statistics
```

Then, you can access the module's functions with "`statistics.`" followed by the name of the function to call. The following calculates the `grades` list's mean, median and mode, using the `statistics` module's **`mean`**, **`median`** and **`mode`** functions:

```
In [4]: statistics.mean(grades)
Out[4]: 79.4

In [5]: statistics.median(grades)
Out[5]: 85

In [6]: statistics.mode(grades)
Out[6]: 85
```

Each function's argument must be an *iterable*—in this case, the list `grades`. To confirm that the median and mode are correct, you can use the built-in **`sorted` function** to get a copy of `grades` with its values arranged in increasing order:

```
In [7]: sorted(grades)
Out[7]: [45, 85, 85, 89, 93]
```

The `grades` list has an odd number of values (5), so `median` returns the middle value (`85`). If the list's number of values is even, `median` returns the *average* of the *two* middle values. Studying the sorted values, you can see that `85` is the mode because it occurs most frequently (twice). The `mode` function causes a `StatisticsError` for lists like

```
[85, 93, 45, 89, 85, 93]
```

in which there are two or more "most frequent" values. Such a set of values is said to be **bimodal**. Here, both `85` and `93` occur twice. We'll say more about mean, median and mode in the Intro to Data Science exercises at the end of the chapter.

Self Check

1 *(Fill-In)* The _____ statistic indicates the average value in a set of values.
Answer: mean.

2 *(Fill-In)* The _____ statistic indicates the most frequently occurring value in a set of values.
Answer: mode.

3 *(Fill-In)* The _____ statistic indicates the middle value in a set of values.
Answer: median.

4 *(IPython Session)* For the values 47, 95, 88, 73, 88 and 84, use the `statistics` module to calculate the mean, median and mode.
Answer:

```
In [1]: import statistics

In [2]: values = [47, 95, 88, 73, 88, 84]

In [3]: statistics.mean(values)
Out[3]: 79.16666666666667

In [4]: statistics.median(values)
Out[4]: 86.0

In [5]: statistics.mode(values)
Out[5]: 88
```

3.18 Wrap-Up

In this chapter, we discussed Python's control statements, including `if`, `if...else`, `if...elif...else`, `while`, `for`, `break` and `continue`. We used pseudocode and top-down, stepwise refinement to develop several algorithms. You saw that many simple algorithms often have three execution phases—initialization, processing and termination.

You saw that the `for` statement performs sequence-controlled iteration—it processes each item in an iterable, such as a range of integers, a string or a list. You used the built-in function `range` to generate sequences of integers from `0` up to, but not including, its argument, and to determine how many times a `for` statement iterates. You used sentinel-controlled repetition with the `while` statement to create a loop that continues executing until a sentinel value is encountered. You used built-in function `range`'s two-argument version to generate sequences of integers from the first argument's value up to, but not including, the second argument's value. You also used the three-argument version in which the third argument indicated the step between integers in a range.

We introduced the `Decimal` type for precise monetary calculations and used it to calculate compound interest. You used f-strings and various format specifiers to create formatted output. We introduced the `break` and `continue` statements for altering the flow of control in loops. We discussed the Boolean operators `and`, `or` and `not` for creating conditions that combine simple conditions.

Finally, we continued our discussion of descriptive statistics by introducing measures of central tendency—mean, median and mode—and calculating them with functions from the Python Standard Library's `statistics` module.

In the next chapter, you'll create custom functions and use existing functions from Python's `math` and `random` modules. We show several predefined functional-programming reductions. You'll learn more of Python's functional-programming capabilities.

Exercises

Unless specified otherwise, use IPython sessions for each exercise.

3.1 *(Validating User Input)* Modify the script of Fig. 3.3 to validate its inputs. For any input, if the value entered is other than 1 or 2, keep looping until the user enters a correct

value. Use one counter to keep track of the number of passes, then calculate the number of failures after all the user's inputs have been received.

3.2 *(What's Wrong with This Code?)* What is wrong with the following code?

```
a = b = 7
print('a =', a, '\nb =', b)
```

First, answer the question, then check your work in an IPython session.

3.3 *(What Does This Code Do?)* What does the following program print?

```
for row in range(10):
    for column in range(10):
        print('<' if row % 2 == 1 else '>', end='')
    print()
```

3.4 *(Fill in the Missing Code)* In the code below

```
for ***:
    for ***:
        print('@')
    print()
```

replace the *** so that when you execute the code, it displays two rows, each containing seven @ symbols, as in:

```
@@@@@@@
@@@@@@@
```

3.5 *(`if...else` Statements)* Reimplement the script of Fig. 2.1 using three `if...else` statements rather than six `if` statements. [*Hint*: For example, think of == and != as "opposite" tests.]

3.6 *(Turing Test)* The great British mathematician Alan Turing proposed a simple test to determine whether machines could exhibit intelligent behavior. A user sits at a computer and does the same text chat with a human sitting at a computer and a computer operating by itself. The user doesn't know if the responses are coming back from the human or the independent computer. If the user can't distinguish which responses are coming from the human and which are coming from the computer, then it's reasonable to say that the computer is exhibiting intelligence.

Create a script that plays the part of the independent computer, giving its user a simple medical diagnosis. The script should prompt the user with `'What is your problem?'` When the user answers and presses *Enter*, the script should simply ignore the user's input, then prompt the user again with `'Have you had this problem before (yes or no)?'` If the user enters `'yes'`, print `'Well, you have it again.'` If the user answers `'no'`, print `'Well, you have it now.'`

Would this conversation convince the user that the entity at the other end exhibited intelligent behavior? Why or why not?

3.7 *(Table of Squares and Cubes)* In Exercise 2.8, you wrote a script to calculate the squares and cubes of the numbers from 0 through 5, then printed the resulting values in table format. Reimplement your script using a `for` loop and the f-string capabilities you learned in this chapter to produce the following table with the numbers right aligned in each column.

```
number  square  cube
     0       0     0
     1       1     1
     2       4     8
     3       9    27
     4      16    64
     5      25   125
```

3.8 *(Arithmetic, Smallest and Largest)* In Exercise 2.10, you wrote a script that input three integers, then displayed the sum, average, product, smallest and largest of those values. Reimplement your script with a loop that inputs four integers.

3.9 *(Separating the Digits in an Integer)* In Exercise 2.11, you wrote a script that separated a five-digit integer into its individual digits and displayed them. Reimplement your script to use a loop that in each iteration "picks off" one digit (left to right) using the `//` and `%` operators, then displays that digit.

3.10 *(7% Investment Return)* Reimplement Exercise 2.12 to use a loop that calculates and displays the amount of money you'll have each year at the ends of years 1 through 30.

3.11 *(Miles Per Gallon)* Drivers are concerned with the mileage obtained by their automobiles. One driver has kept track of several tankfuls of gasoline by recording miles driven and gallons used for each tankful. Develop a sentinel-controlled-repetition script that prompts the user to input the miles driven and gallons used for each tankful. The script should calculate and display the miles per gallon obtained for each tankful. After processing all input information, the script should calculate and display the combined miles per gallon obtained for all tankfuls (that is, total miles driven divided by total gallons used).

```
Enter the gallons used (-1 to end): 12.8
Enter the miles driven: 287
The miles/gallon for this tank was 22.421875
Enter the gallons used (-1 to end): 10.3
Enter the miles driven: 200
The miles/gallon for this tank was 19.417475
Enter the gallons used (-1 to end): 5
Enter the miles driven: 120
The miles/gallon for this tank was 24.000000
Enter the gallons used (-1 to end): -1
The overall average miles/gallon was 21.601423
```

3.12 *(Palindromes)* A palindrome is a number, word or text phrase that reads the same backwards or forwards. For example, each of the following five-digit integers is a palindrome: 12321, 55555, 45554 and 11611. Write a script that reads in a five-digit integer and determines whether it's a palindrome. [*Hint*: Use the `//` and `%` operators to separate the number into its digits.]

3.13 *(Factorials)* Factorial calculations are common in probability. The factorial of a nonnegative integer n is written $n!$ (pronounced "n factorial") and is defined as follows:

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \ldots \cdot 1$$

for values of n greater than or equal to 1, with 0! defined to be 1. So,

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

which is 120. Factorials increase in size very rapidly. Write a script that inputs a nonnegative integer and computes and displays its factorial. Try your script on the integers 10, 20,

30 and even larger values. Did you find any integer input for which Python could not produce an integer factorial value?

3.14 ***(Challenge: Approximating the Mathematical Constant π)*** Write a script that computes the value of π from the following infinite series. Print a table that shows the value of π approximated by one term of this series, by two terms, by three terms, and so on. How many terms of this series do you have to use before you first get 3.14? 3.141? 3.1415? 3.14159?

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \cdots$$

3.15 ***(Challenge: Approximating the Mathematical Constant e)*** Write a script that estimates the value of the mathematical constant *e* by using the formula below. Your script can stop after summing 10 terms.

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots$$

3.16 ***(Nested Control Statements)*** Use a loop to find the *two* largest values of 10 numbers entered.

3.17 ***(Nested Loops)*** Write a script that displays the following triangle patterns separately, one below the other. Separate each pattern from the next by one blank line. Use `for` loops to generate the patterns. Display all asterisks (*) with a single statement of the form

```
print('*', end='')
```

which causes the asterisks to display side by side. [*Hint*: For the last two patterns, begin each line with zero or more space characters.]

```
(a)          (b)          (c)          (d)
*            **********   **********            *
**           *********     *********           **
***          ********       ********         ***
****         *******         *******       ****
*****        ******           ******     *****
******       *****             *****   ******
*******      ****               **** *******
********     ***                 ********
*********    **                   ******
**********   *                     ****
```

3.18 ***(Challenge: Nested Looping)*** Modify your script from Exercise 3.17 to display all four patterns side-by-side (as shown above) by making clever use of nested `for` loops. Separate each triangle from the next by three horizontal spaces. [*Hint*: One `for` loop should control the row number. Its nested `for` loops should calculate from the row number the appropriate number of asterisks and spaces for each of the four patterns.]

3.19 ***(Brute-Force Computing: Pythagorean Triples)*** A right triangle can have sides that are all integers. The set of three integer values for the sides of a right triangle is called a Pythagorean triple. These three sides must satisfy the relationship that the sum of the squares of two of the sides is equal to the square of the hypotenuse. Find all Pythagorean triples for `side1`, `side2` and `hypotenuse` (such as 3, 4 and 5) all no larger than 20. Use a triple-nested `for`-loop that tries all possibilities. This is an example of "brute-force" computing. You'll

learn in more advanced computer science courses that there are many interesting problems for which there is no known algorithmic approach other than sheer brute force.

3.20 *(Binary-to-Decimal Conversion)* Input an integer containing 0s and 1s (i.e., a "binary" integer) and display its decimal equivalent. The online appendix, "Number Systems," discusses the binary number system. [*Hint*: Use the modulus and division operators to pick off the "binary" number's digits one at a time from right to left. Just as in the decimal number system, where the rightmost digit has the positional value 1 and the next digit to the left has the positional value 10, then 100, then 1000, etc., in the binary number system, the rightmost digit has the positional value 1, the next digit to the left has the positional value 2, then 4, then 8, etc. Thus, the decimal number 234 can be interpreted as 2 * 100 + 3 * 10 + 4 * 1. The decimal equivalent of binary 1101 is 1 * 8 + 1 * 4 + 0 * 2 + 1 * 1.]

3.21 *(Calculate Change Using Fewest Number of Coins)* Write a script that inputs a purchase price of a dollar or less for an item. Assume the purchaser pays with a dollar bill. Determine the amount of change the cashier should give back to the purchaser. Display the change using the *fewest* number of pennies, nickels, dimes and quarters. For example, if the purchaser is due 73 cents in change, the script would output:

```
Your change is:
2 quarters
2 dimes
3 pennies
```

3.22 *(Optional `else` Clause of a Loop)* The `while` and `for` statements each have an optional `else` clause. In a `while` statement, the `else` clause executes when the condition becomes `False`. In a `for` statement, the `else` clause executes when there are no more items to process. If you `break` out of a `while` or `for` that has an `else`, the `else` part does *not* execute. Execute the following code to see that the `else` clause executes only if the `break` statement does not:

```
for i in range(2):
    value = int(input('Enter an integer (-1 to break): '))
    print('You entered:', value)

    if value == -1:
        break
else:
    print('The loop terminated without executing the break')
```

For more information on loop `else` clauses, see

```
https://docs.python.org/3/tutorial/controlflow.html#break-and-
continue-statements-and-else-clauses-on-loops
```

3.23 *(Validating Indentation)* The file `validate_indents.py` in this chapter's `ch03` examples folder contains the following code with incorrect indentation:

```
grade = 93

if grade >= 90:
    print('A')
  print('Great Job!')
   print('Take a break from studying')
```

The Python Standard Library includes a code indentation validator module named **tabnanny**, which you can run as a script to check your code for proper indentation—this is one of many static code analysis tools. Execute the following command in the `ch03` folder to see the results of analyzing `validate_indents.py`:

```
python -m tabnanny validate_indents.py
```

Suppose you accidentally aligned the second `print` statement under the `i` in the `if` keyword. What kind of error would that be? Would you expect `tabnanny` to flag that as an error?

3.24 *(Project: Using the **prospector** Static Code Analysis Tool)* The `prospector` tool runs several popular static code analysis tools to check your Python code for common errors and to help you improve your code. Check that you've installed `prospector` (see the Before You Begin section that follows the Preface). Run prospector on each of the scripts in this chapter. To do so, open the folder containing the scripts in a Terminal (macOS/Linux), Command Prompt (Windows) or shell (Linux), then run the following command from that folder:

```
prospector --strictness veryhigh --doc-warnings
```

Study the output to see the kinds of issues prospector locates in Python code. In general, run `prospector` on all new code you create.

3.25 *(Project: Using **prospector** to Analyze Open-Source Code on GitHub)* Locate a Python open-source project on GitHub, download its source code and extract it into a folder on your system. Open that folder in a Terminal (macOS/Linux), Command Prompt (Windows) or shell (Linux), then run the following command from that folder:

```
prospector --strictness veryhigh --doc-warnings
```

Study the output to see more of the kinds of issues prospector locates in Python code.

3.26 *(Research: Anscombe's Quartet)* In this book's data science case studies, we'll emphasize the importance of "getting to know your data." The *basic descriptive statistics* that you've seen in this chapter's and the previous chapter's Intro to Data Science sections certainly help you know more about your data. One caution, though, is that different datasets can have identical or nearly identical descriptive statistics and yet the data can be significantly different. For an example of this phenomenon, research *Anscombe's Quartet*. You should find four datasets and the associated visualizations. It's the visualizations that convince you the datasets are quite different. In an exercise in a later chapter, you'll create these visualizations.

3.27 *(World Population Growth)* World population has grown considerably over the centuries. Continued growth could eventually challenge the limits of breathable air, drinkable water, arable land and other limited resources. There's evidence that growth has been slowing in recent years and that world population could peak some time this century, then start to decline.

For this exercise, research world population growth issues. This is a controversial topic, so be sure to investigate various viewpoints. Get estimates for the current world population and its growth rate. Write a script that calculates world population growth each year for the next 100 years, *using the simplifying assumption that the current growth rate will stay constant*. Print the results in a table. The first column should display the year

from 1 to 100. The second column should display the anticipated world population at the end of that year. The third column should display the numerical increase in the world population that would occur that year. Using your results, determine the years in which the population would be double and eventually quadruple what it is today.

3.28 ***(Intro to Data Science: Mean, Median and Mode)*** Calculate the mean, median and mode of the values 9, 11, 22, 34, 17, 22, 34, 22 and 40. Suppose the values included another 34. What problem might occur?

3.29 ***(Intro to Data Science: Problem with the Median)*** For an odd number of values, to get the median you simply arrange them in order and take the middle value. For an even number, you average the two middle values. What problem occurs if those two values are different?

3.30 ***(Intro to Data Science: Outliers)*** In statistics, outliers are values out of the ordinary and possibly way out of the ordinary. Sometimes, outliers are simply bad data. In the data science case studies, we'll see that outliers can distort results. Which of the three measures of central tendency we discussed—mean, median and mode—is most affected by outliers? Why? Which of these measures are not affected or least affected? Why?

3.31 ***(Intro to Data Science: Categorical Data)*** Mean, median and mode work well with numerical values. You can use them in calculations and arrange them in meaningful order. Categorical values are descriptive names like Boxer, Poodle, Collie, Beagle, Bulldog and Chihuahua. Normally, you don't use these in calculations nor associate an order with them. Which if any of the descriptive statistics are appropriate for categorical data?

4

Functions

Objectives

In this chapter, you'll

- Create custom functions.
- Import and use Python Standard Library modules, such as `random` and `math`, to reuse code and avoid "reinventing the wheel."
- Pass data between functions.
- Generate a range of random numbers.
- Learn simulation techniques using random-number generation.
- Pack values into a tuple and unpack values from a tuple.
- Return multiple values from a function via a tuple.
- Understand how an identifier's scope determines where in your program you can use it.
- Create functions with default parameter values.
- Call functions with keyword arguments.
- Create functions that can receive any number of arguments.
- Use methods of an object.

4.1 Introduction

Experience has shown that the best way to develop and maintain a large program is to construct it from smaller, more manageable pieces. This technique is called **divide and conquer**. Using existing functions as building blocks for creating new programs is a key aspect of **software reusability**—it's also a major benefit of object-oriented programming. Packaging code as a function allows you to execute it from various locations in your program just by calling the function, rather than duplicating the possibly lengthy code. This also makes programs easier to modify. When you change a function's code, all calls to the function execute the updated version.

4.2 Defining Functions

You've called many built-in functions (`int`, `float`, `print`, `input`, `type`, `sum`, `len`, `min` and `max`) and a few functions from the statistics module (`mean`, `median` and `mode`). Each performed a single, well-defined task. You'll often define and call *custom* functions. The following session defines a `square` function that calculates the square of its argument. Then it calls the function twice—once to square the `int` value `7` (producing the `int` value `49`) and once to square the `float` value `2.5` (producing the `float` value `6.25`):

```
In [1]: def square(number):
   ...:     """Calculate the square of number."""
   ...:     return number ** 2
   ...:

In [2]: square(7)
Out[2]: 49

In [3]: square(2.5)
Out[3]: 6.25
```

The statements defining the function in the first snippet are written only once, but may be called "to do their job" from many points throughout a program and as often as you like. Calling `square` with a non-numeric argument like `'hello'` causes a `TypeError` because the exponentiation operator (`**`) works only with numeric values.

Defining a Custom Function

A **function definition** (like `square` in snippet `[1]`) begins with the **`def` keyword**, followed by the function name (`square`), a set of parentheses and a colon (`:`). Like variable identifiers, by convention function names should begin with a lowercase letter and in multiword names underscores should separate each word.

The required parentheses contain the function's **parameter list**—a comma-separated list of **parameters** representing the data that the function needs to perform its task. Function `square` has only one parameter named `number`—the value to be squared.

If the parentheses are empty, the function does not use parameters to perform its task. Exercise 4.7 asks you to write a parameterless `date_and_time` function that displays the current date and time by reading it from your computer's system clock.

The indented lines after the colon (`:`) are the function's **block**, which consists of an optional docstring followed by the statements that perform the function's task. We'll soon point out the difference between a function's block and a control statement's suite.

Specifying a Custom Function's Docstring

The *Style Guide for Python Code* says that the first line in a function's block should be a docstring that briefly explains the function's purpose:

```
"""Calculate the square of number."""
```

To provide more detail, you can use a multiline docstring—the style guide recommends starting with a brief explanation, followed by a blank line and the additional details.

Returning a Result to a Function's Caller

When a function finishes executing, it returns control to its caller—that is, the line of code that called the function. In `square`'s block, the **`return`** statement:

```
return number ** 2
```

first squares `number`, then terminates the function and gives the result back to the caller. In this example, the first caller is `square(7)` in snippet `[2]`, so IPython displays the result in `Out[2]`. Think of the return value, `49`, as simply replacing the call `square(7)`. So after the call, you'd have `In [2]: 49`, and that would indeed produce `Out[2]: 49`. The second caller `square(2.5)` is in snippet `[3]`, so IPython displays the result `6.25` in `Out[3]`.

Function calls also can be embedded in expressions. The following code calls `square` first, then `print` displays the result:

```
In [4]: print('The square of 7 is', square(7))
The square of 7 is 49
```

Here, too, think of the return value, `49`, as simply replacing the call `square(7)`, which would indeed produce the output shown above.

There are two other ways to return control from a function to its caller:

- Executing a `return` statement without an expression terminates the function and *implicitly* returns the value **`None`** to the caller. The Python documentation states that `None` represents the absence of a value. `None` evaluates to `False` in conditions.
- When there's no `return` statement in a function, it *implicitly* returns the value `None` after executing the last statement in the function's block.

What Happens When You Call a Function

The expression `square(7)` passes the argument `7` to `square`'s parameter `number`. Then `square` calculates `number ** 2` and returns the result. The parameter `number` exists only during the function call. It's created on each call to the function to receive the argument value, and it's destroyed when the function returns its result to the caller.

Though we did not define variables in `square`'s block, it is possible to do so. A function's parameters and variables defined in its block are all **local variables**—they can be used only inside the function and exist only while the function is executing. Trying to access a local variable outside its function's block causes a `NameError`, indicating that the variable is not defined. We'll soon see how a behind-the-scenes mechanism called the *function-call stack* supports the automatic creation and destruction of a function's local variables—and helps the function return to its caller.

Accessing a Function's Docstring via IPython's Help Mechanism

IPython can help you learn about the modules and functions you intend to use in your code, as well as IPython itself. For example, to view a function's docstring to learn how to use the function, type the function's name followed by a **question mark (?)**:

```
In [5]: square?
Signature: square(number)
Docstring: Calculate the square of number.
File:      ~/Documents/examples/ch04/<ipython-input-1-7268c8ff93a9>
Type:      function
```

For our `square` function, the information displayed includes:

- The function's name and parameter list—known as its **signature**.
- The function's docstring.
- The name of the file containing the function's definition. For a function in an interactive session, this line shows information for the snippet that defined the function—the `1` in `"<ipython-input-1-7268c8ff93a9>"` means snippet `[1]`.
- The type of the item for which you accessed IPython's help mechanism—in this case, a function.

If the function's source code is accessible from IPython—such as a function defined in the current session or imported into the session from a `.py` file—you can use **??** to display the function's full source-code definition:

```
In [6]: square??
Signature: square(number)
Source:
def square(number):
    """Calculate the square of number."""
    return number ** 2
File:      ~/Documents/examples/ch04/<ipython-input-1-7268c8ff93a9>
Type:      function
```

If the source code is not accessible from IPython, `??` simply shows the docstring.

If the docstring fits in the window, IPython displays the next `In []` prompt. If a docstring is too long to fit, IPython indicates that there's more by displaying a colon (`:`) at the bottom of the window—press the *Space* key to display the next screen. You can navigate

backwards and forwards through the docstring with the up and down arrow keys, respectively. IPython displays `(END)` at the end of the docstring. Press *q* (for "quit") at any `:` or the `(END)` prompt to return to the next `In []` prompt. To get a sense of IPython's features, type `?` at any `In []` prompt, press *Enter*, then read the help documentation overview.

Self Check

1 *(True/False)* The function body is referred to as its suite.
Answer: False. The function body is referred to as its block.

2 *(True/False)* A function's local variables exist after the function returns to its caller.
Answer: False. A function's local variables exist until the function returns to its caller.

3 *(IPython Session)* Define a function `square_root` that receives a number as a parameter and returns the square root of that number. Determine the square root of `6.25`.
Answer:

```
In [1]: def square_root(number):
   ...:     return number ** 0.5  # or number ** (1 / 2)
   ...:

In [2]: square_root(6.25)
Out[2]: 2.5
```

4.3 Functions with Multiple Parameters

Let's define a `maximum` function that determines and returns the largest of three values—the following session calls the function three times with integers, floating-point numbers and strings, respectively.

```
In [1]: def maximum(value1, value2, value3):
   ...:     """Return the maximum of three values."""
   ...:     max_value = value1
   ...:     if value2 > max_value:
   ...:         max_value = value2
   ...:     if value3 > max_value:
   ...:         max_value = value3
   ...:     return max_value
   ...:

In [2]: maximum(12, 27, 36)
Out[2]: 36

In [3]: maximum(12.3, 45.6, 9.7)
Out[3]: 45.6

In [4]: maximum('yellow', 'red', 'orange')
Out[4]: 'yellow'
```

We did not place blank lines above and below the `if` statements, because pressing return on a blank line in interactive mode completes the function's definition.

You also may call `maximum` with mixed types, such as `int`s and `float`s:

```
In [5]: maximum(13.5, -3, 7)
Out[5]: 13.5
```

The call `maximum(13.5, 'hello', 7)` results in `TypeError` because strings and numbers cannot be compared to one another with the greater-than (>) operator.

Function `maximum`'s Definition

Function `maximum` specifies three parameters in a comma-separated list. Snippet `[2]`'s arguments `12`, `27` and `36` are assigned to the parameters `value1`, `value2` and `value3`, respectively.

To determine the largest value, we process one value at a time:

- Initially, we assume that `value1` contains the largest value, so we assign it to the local variable `max_value`. Of course, it's possible that `value2` or `value3` contains the actual largest value, so we still must compare each of these with `max_value`.
- The first `if` statement then tests `value2 > max_value`, and if this condition is `True` assigns `value2` to `max_value`.
- The second `if` statement then tests `value3 > max_value`, and if this condition is `True` assigns `value3` to `max_value`.

Now, `max_value` contains the largest value, so we return it. When control returns to the caller, the parameters `value1`, `value2` and `value3` and the variable `max_value` in the function's block—which are all *local variables*—no longer exist.

Python's Built-In `max` and `min` Functions

For many common tasks, the capabilities you need already exist in Python. For example, built-in **`max`** and **`min`** functions know how to determine the largest and smallest of their two or more arguments, respectively:

```
In [6]: max('yellow', 'red', 'orange', 'blue', 'green')
Out[6]: 'yellow'

In [7]: min(15, 9, 27, 14)
Out[7]: 9
```

Each of these functions also can receive an iterable argument, such as a list or a string. Using built-in functions or functions from the Python Standard Library's modules rather than writing your own can reduce development time and increase program reliability, portability and performance. For a list of Python's built-in functions and modules, see

```
https://docs.python.org/3/library/index.html
```

Self Check

1 *(Fill-In)* A function with multiple parameters specifies them in a(n) _____.
Answer: comma-separated list.

2 *(True/False)* When defining a function in IPython interactive mode, pressing *Enter* on a blank line causes IPython to display another continuation prompt so you can continue defining the function's block.
Answer: False. When defining a function in IPython interactive mode, pressing *Enter* on a blank line terminates the function definition.

3 *(IPython Session)* Call function `max` with the list `[14, 27, 5, 3]` as an argument, then call function `min` with the string `'orange'` as an argument.

Answer:

```
In [1]: max([14, 27, 5, 3])
Out[1]: 27

In [2]: min('orange')
Out[2]: 'a'
```

4.4 Random-Number Generation

We now take a brief diversion into a popular type of programming application—simulation and game playing. You can introduce the **element of chance** via the Python Standard Library's **`random` module**.

Rolling a Six-Sided Die

Let's produce 10 random integers in the range 1–6 to simulate rolling a six-sided die:

```
In [1]: import random

In [2]: for roll in range(10):
   ...:     print(random.randrange(1, 7), end=' ')
   ...:
4 2 5 5 4 6 4 6 1 5
```

First, we import `random` so we can use the module's capabilities. The **`randrange`** function generates an integer from the first argument value up to, but *not* including, the second argument value. Let's use the up arrow key to recall the `for` statement, then press *Enter* to re-execute it. Notice that *different* values are displayed:

```
In [3]: for roll in range(10):
   ...:     print(random.randrange(1, 7), end=' ')
   ...:
4 5 4 5 1 4 1 4 6 5
```

Sometimes, you may want to guarantee **reproducibility** of a random sequence—for debugging, for example. At the end of this section, we'll show how to do this with the `random` module's `seed` function.

Rolling a Six-Sided Die 6,000,000 Times

If `randrange` truly produces integers at random, every number in its range has an equal **probability** (or *chance* or *likelihood*) of being returned each time we call it. To show that the die faces 1–6 occur with equal likelihood, the following script simulates 6,000,000 die rolls. When you run the script, each die face should occur approximately 1,000,000 times, as in the sample output.

```
1  # fig04_01.py
2  """Roll a six-sided die 6,000,000 times."""
3  import random
4
5  # face frequency counters
6  frequency1 = 0
```

Fig. 4.1 | Roll a six-sided die 6,000,000 times. (Part 1 of 2.)

```
frequency2 = 0
frequency3 = 0
frequency4 = 0
frequency5 = 0
frequency6 = 0

# 6,000,000 die rolls
for roll in range(6_000_000):  # note underscore separators
    face = random.randrange(1, 7)

    # increment appropriate face counter
    if face == 1:
        frequency1 += 1
    elif face == 2:
        frequency2 += 1
    elif face == 3:
        frequency3 += 1
    elif face == 4:
        frequency4 += 1
    elif face == 5:
        frequency5 += 1
    elif face == 6:
        frequency6 += 1

print(f'Face{"Frequency":>13}')
print(f'{1:>4}{frequency1:>13}')
print(f'{2:>4}{frequency2:>13}')
print(f'{3:>4}{frequency3:>13}')
print(f'{4:>4}{frequency4:>13}')
print(f'{5:>4}{frequency5:>13}')
print(f'{6:>4}{frequency6:>13}')
```

```
Face    Frequency
   1       998686
   2      1001481
   3       999900
   4      1000453
   5       999953
   6       999527
```

Fig. 4.1 | Roll a six-sided die 6,000,000 times. (Part 2 of 2.)

The script uses *nested* control statements (an `if...elif` statement nested in the `for` statement) to determine the number of times each die face appears. The `for` statement iterates 6,000,000 times. We used Python's underscore (_) digit separator to make the value `6000000` more readable. The expression `range(6,000,000)` would be incorrect. Commas separate arguments in function calls, so Python would treat `range(6,000,000)` as a call to `range` with the *three* arguments `6`, `0` and `0`.

For each die roll, the script adds `1` to the appropriate counter variable. Run the program, and observe the results. This program might take a few seconds to complete execution. As you'll see, each execution produces *different* results.

Note that we did not provide an `else` clause in the `if...elif` statement. Exercise 4.1 asks you to comment on the possible consequences of this.

Seeding the Random-Number Generator for Reproducibility

Function `randrange` actually generates **pseudorandom numbers**, based on an internal calculation that begins with a numeric value known as a **seed**. Repeatedly calling `randrange` produces a sequence of numbers that *appear* to be random, because each time you start a new interactive session or execute a script that uses the `random` module's functions, Python internally uses a *different* seed value.[1] When you're debugging logic errors in programs that use randomly generated data, it can be helpful to use the *same* sequence of random numbers until you've eliminated the logic errors, before testing the program with other values. To do this, you can use the random module's **seed** function to **seed the random-number generator** yourself—this forces `randrange` to begin calculating its pseudorandom number sequence from the seed you specify. In the following session, snippets `[5]` and `[8]` produce the same results, because snippets `[4]` and `[7]` use the same seed (`32`):

```
In [4]: random.seed(32)

In [5]: for roll in range(10):
   ...:     print(random.randrange(1, 7), end=' ')
   ...:
1 2 2 3 6 2 4 1 6 1
In [6]: for roll in range(10):
   ...:     print(random.randrange(1, 7), end=' ')
   ...:
1 3 5 3 1 5 6 4 3 5
In [7]: random.seed(32)

In [8]: for roll in range(10):
   ...:     print(random.randrange(1, 7), end=' ')
   ...:
1 2 2 3 6 2 4 1 6 1
```

Snippet `[6]` generates *different* values because it simply continues the pseudorandom number sequence that began in snippet `[5]`.

Self Check

1 *(Fill-In)* The element of chance can be introduced into computer applications using module _____.

Answer: `random`.

2 *(Fill-In)* The `random` module's _____ function enables reproducibility of random sequences.

Answer: `seed`.

1. According to the documentation, Python bases the seed value on the system clock or an operating-system-dependent randomness source. For applications requiring secure random numbers, such as cryptography, the documentation recommends using the `secrets` module, rather than the `random` module.

3 *(IPython Session)* Requirements statement: Use a `for` statement, `randrange` and a conditional expression (introduced in the preceding chapter) to simulate 20 coin flips, displaying `H` for heads and `T` for tails all on the same line, each separated by a space.
Answer:

```
In [1]: import random

In [2]: for i in range(20):
   ...:     print('H' if random.randrange(2) == 0 else 'T', end=' ')
   ...:
T H T T H T T T T H T H H T H T H H H H
```

In snippet `[2]`'s output, an equal number of `T`s and `H`s appeared—that will not always be the case with random-number generation.

4.5 Case Study: A Game of Chance

In this section, we simulate the popular dice game known as "craps." Here is the requirements statement:

> *You roll two six-sided dice, each with faces containing one, two, three, four, five and six spots, respectively. When the dice come to rest, the sum of the spots on the two upward faces is calculated. If the sum is 7 or 11 on the first roll, you win. If the sum is 2, 3 or 12 on the first roll (called "craps"), you lose (i.e., the "house" wins). If the sum is 4, 5, 6, 8, 9 or 10 on the first roll, that sum becomes your "point." To win, you must continue rolling the dice until you "make your point" (i.e., roll that same point value). You lose by rolling a 7 before making your point.*

The following script simulates the game and shows several sample executions, illustrating winning on the first roll, losing on the first roll, winning on a subsequent roll and losing on a subsequent roll.

```
# fig04_02.py
"""Simulating the dice game Craps."""
import random

def roll_dice():
    """Roll two dice and return their face values as a tuple."""
    die1 = random.randrange(1, 7)
    die2 = random.randrange(1, 7)
    return (die1, die2)  # pack die face values into a tuple

def display_dice(dice):
    """Display one roll of the two dice."""
    die1, die2 = dice  # unpack the tuple into variables die1 and die2
    print(f'Player rolled {die1} + {die2} = {sum(dice)}')

die_values = roll_dice()  # first roll
display_dice(die_values)

```

Fig. 4.2 | Game of craps. (Part 1 of 2.)

```
# determine game status and point, based on first roll
sum_of_dice = sum(die_values)

if sum_of_dice in (7, 11):  # win
    game_status = 'WON'
elif sum_of_dice in (2, 3, 12):  # lose
    game_status = 'LOST'
else:  # remember point
    game_status = 'CONTINUE'
    my_point = sum_of_dice
    print('Point is', my_point)

# continue rolling until player wins or loses
while game_status == 'CONTINUE':
    die_values = roll_dice()
    display_dice(die_values)
    sum_of_dice = sum(die_values)

    if sum_of_dice == my_point:  # win by making point
        game_status = 'WON'
    elif sum_of_dice == 7:  # lose by rolling 7
        game_status = 'LOST'

# display "wins" or "loses" message
if game_status == 'WON':
    print('Player wins')
else:
    print('Player loses')
```

```
Player rolled 2 + 5 = 7
Player wins
```

```
Player rolled 1 + 2 = 3
Player loses
```

```
Player rolled 5 + 4 = 9
Point is 9
Player rolled 4 + 4 = 8
Player rolled 2 + 3 = 5
Player rolled 5 + 4 = 9
Player wins
```

```
Player rolled 1 + 5 = 6
Point is 6
Player rolled 1 + 6 = 7
Player loses
```

Fig. 4.2 | Game of craps. (Part 2 of 2.)

Function `roll_dice`—Returning Multiple Values Via a Tuple

Function `roll_dice` (lines 5–9) simulates rolling two dice on each roll. The function is defined once, then called from several places in the program (lines 16 and 33). The empty parameter list indicates that `roll_dice` does not require arguments to perform its task.

The built-in and custom functions you've called so far each return one value. Sometimes it's useful to return more than one value, as in `roll_dice`, which returns both die values (line 9) as a **tuple**—an **immutable** (that is, unmodifiable) sequences of values. To create a tuple, separate its values with commas, as in line 9:

```
(die1, die2)
```

This is known as **packing a tuple**. The parentheses are optional, but we recommend using them for clarity. We discuss tuples in depth in the next chapter.

Function `display_dice`

To use a tuple's values, you can assign them to a comma-separated list of variables, which **unpacks** the tuple. To display each roll of the dice, the function `display_dice` (defined in lines 11–14 and called in lines 17 and 34) unpacks the tuple argument it receives (line 13). The number of variables to the left of `=` must match the number of elements in the tuple; otherwise, a `ValueError` occurs. Line 14 `print`s a formatted string containing both die values and their sum. We calculate the sum of the dice by passing the tuple to the built-in `sum` function—like a list, a tuple is a sequence.

Note that functions `roll_dice` and `display_dice` each begin their blocks with a docstring that states what the function does. Also, both functions contain local variables `die1` and `die2`. These variables do not "collide," because they belong to different functions' blocks. Each local variable is accessible only in the block that defined it.

First Roll

When the script begins executing, lines 16–17 roll the dice and display the results. Line 20 calculates the sum of the dice for use in lines 22–29. You can win or lose on the first roll or any subsequent roll. The variable `game_status` keeps track of the win/loss status.

The **`in` operator** in line 22

```
sum_of_dice in (7, 11)
```

tests whether the tuple `(7, 11)` contains `sum_of_dice`'s value. If this condition is `True`, you rolled a `7` or an `11`. In this case, you won on the first roll, so the script sets `game_status` to `'WON'`. The operator's right operand can be any iterable. There's also a **`not in`** operator to determine whether a value is *not* in an iterable. The preceding concise condition is equivalent to

```
(sum_of_dice == 7) or (sum_of_dice == 11)
```

Similarly, the condition in line 24

```
sum_of_dice in (2, 3, 12)
```

tests whether the tuple `(2, 3, 12)` contains `sum_of_dice`'s value. If so, you lost on the first roll, so the script sets `game_status` to `'LOST'`.

For any other sum of the dice (`4`, `5`, `6`, `8`, `9` or `10`):

- line 27 sets `game_status` to `'CONTINUE'` so you can continue rolling

- line 28 stores the sum of the dice in `my_point` to keep track of what you must roll to win and
- line 29 displays `my_point`.

Subsequent Rolls

If `game_status` is equal to `'CONTINUE'` (line 32), you did not win or lose, so the `while` statement's suite (lines 33–40) executes. Each loop iteration calls `roll_dice`, displays the die values and calculates their sum. If `sum_of_dice` is equal to `my_point` (line 37) or `7` (line 39), the script sets `game_status` to `'WON'` or `'LOST'`, respectively, and the loop terminates. Otherwise, the `while` loop continues executing with the next roll.

Displaying the Final Results

When the loop terminates, the script proceeds to the `if...else` statement (lines 43–46), which prints `'Player wins'` if `game_status` is `'WON'`, or `'Player loses'` otherwise.

Self Check

1 *(Fill-In)* The _____ operator tests whether its right operand's iterable contains its left operand's value.
Answer: `in`.

2 *(IPython Session)* Pack a `student` tuple with the name `'Sue'` and the list `[89, 94, 85]`, display the tuple, then unpack it into variables `name` and `grades`, and display their values.
Answer:

```
In [1]: student = ('Sue', [89, 94, 85])

In [2]: student
Out[2]: ('Sue', [89, 94, 85])

In [3]: name, grades = student

In [4]: print(f'{name}: {grades}')
Sue: [89, 94, 85]
```

4.6 Python Standard Library

Typically, you write Python programs by combining functions and classes (that is, custom types) that you create with preexisting functions and classes defined in modules, such as those in the Python Standard Library and other libraries. A key programming goal is to avoid "reinventing the wheel."

A module is a file that groups related functions, data and classes. The type `Decimal` from the Python Standard Library's `decimal` module is actually a class. We introduced classes briefly in Chapter 1 and discuss them in detail in the "Object-Oriented Programming" chapter. A **package** groups related modules. In this book, you'll work with many preexisting modules and packages, and you'll create your own modules—in fact, every Python source-code (`.py`) file you create is a module. Creating packages is beyond this book's scope. They're typically used to organize a large library's functionality into smaller subsets that are easier to maintain and can be imported separately for convenience. For example, the `matplotlib` visualization library that we use in Section 5.17 has extensive

functionality (its documentation is over 2300 pages), so we'll import only the subsets we need in our examples (`pyplot` and `animation`).

The Python Standard Library is provided with the core Python language. Its packages and modules contain capabilities for a wide variety of everyday programming tasks.[2] You can see a complete list of the standard library modules at

```
https://docs.python.org/3/library/
```

You've already used capabilities from the `decimal`, `statistics` and `random` modules. In the next section, you'll use mathematics capabilities from the `math` module. You'll see many other Python Standard Library modules throughout the book's examples and exercises, including many of those in the following table:

Some popular Python Standard Library modules

`collections`—Data structures beyond lists, tuples, dictionaries and sets.
Cryptography modules—Encrypting data for secure transmission.
`csv`—Processing comma-separated value files (like those in Excel).
`datetime`—Date and time manipulations. Also modules `time` and `calendar`.
`decimal`—Fixed-point and floating-point arithmetic, including monetary calculations.
`doctest`—Embed validation tests and expected results in docstrings for simple unit testing.
`gettext` and `locale`—Internationalization and localization modules.
`json`—JavaScript Object Notation (JSON) processing used with web services and NoSQL document databases.
`math`—Common math constants and operations.
`os`—Interacting with the operating system.
`profile`, `pstats`, `timeit`—Performance analysis.
`random`—Pseudorandom numbers.
`re`—Regular expressions for pattern matching.
`sqlite3`—SQLite relational database access.
`statistics`—Mathematical statistics functions such as `mean`, `median`, `mode` and `variance`.
`string`—String processing.
`sys`—Command-line argument processing; standard input, standard output and standard error streams.
`tkinter`—Graphical user interfaces (GUIs) and canvas-based graphics.
`turtle`—Turtle graphics.
`webbrowser`—For conveniently displaying web pages in Python apps.

Self Check

1 *(Fill-In)* A(n) _____ defines related functions, data and classes. A(n) _____ groups related modules.
Answer: module, package.

2 *(Fill-In)* Every Python source code (`.py`) file you create is a(n) _____.
Answer: module.

4.7 `math` Module Functions

The **`math` module** defines functions for performing various common mathematical calculations. Recall from the previous chapter that an `import` statement of the following form enables you to use a module's definitions via the module's name and a dot (`.`):

```
In [1]: import math
```

2. The Python Tutorial refers to this as the "batteries included" approach.

For example, the following snippet calculates the square root of 900 by calling the `math` module's **`sqrt` function**, which returns its result as a `float` value:

```
In [2]: math.sqrt(900)
Out[2]: 30.0
```

Similarly, the following snippet calculates the absolute value of `-10` by calling the `math` module's **`fabs` function**, which returns its result as a `float` value:

```
In [3]: math.fabs(-10)
Out[3]: 10.0
```

Some `math` module functions are summarized below—you can view the complete list at

`https://docs.python.org/3/library/math.html`

Function	Description	Example
`ceil(`*x*`)`	Rounds *x* to the smallest integer not less than *x*	`ceil(9.2)` is `10.0` `ceil(-9.8)` is `-9.0`
`floor(`*x*`)`	Rounds *x* to the largest integer not greater than *x*	`floor(9.2)` is `9.0` `floor(-9.8)` is `-10.0`
`sin(`*x*`)`	Trigonometric sine of *x* (*x* in radians)	`sin(0.0)` is `0.0`
`cos(`*x*`)`	Trigonometric cosine of *x* (*x* in radians)	`cos(0.0)` is `1.0`
`tan(`*x*`)`	Trigonometric tangent of *x* (*x* in radians)	`tan(0.0)` is `0.0`
`exp(`*x*`)`	Exponential function e^x	`exp(1.0)` is `2.718282` `exp(2.0)` is `7.389056`
`log(`*x*`)`	Natural logarithm of *x* (base *e*)	`log(2.718282)` is `1.0` `log(7.389056)` is `2.0`
`log10(`*x*`)`	Logarithm of *x* (base 10)	`log10(10.0)` is `1.0` `log10(100.0)` is `2.0`
`pow(`*x*`, `*y*`)`	*x* raised to power *y* (x^y)	`pow(2.0, 7.0)` is `128.0` `pow(9.0, .5)` is `3.0`
`sqrt(`*x*`)`	square root of *x*	`sqrt(900.0)` is `30.0` `sqrt(9.0)` is `3.0`
`fabs(`*x*`)`	Absolute value of *x*—always returns a float. Python also has the built-in function `abs`, which returns an `int` or a `float`, based on its argument.	`fabs(5.1)` is `5.1` `fabs(-5.1)` is `5.1`
`fmod(`*x*`, `*y*`)`	Remainder of *x*/*y* as a floating-point number	`fmod(9.8, 4.0)` is `1.8`

4.8 Using IPython Tab Completion for Discovery

You can view a module's documentation in IPython interactive mode via **tab completion**—a **discovery** feature that speeds your coding and learning processes. After you type a portion of an identifier and press *Tab*, IPython completes the identifier for you or provides a list of identifiers that begin with what you've typed so far. This may vary based on your operating system platform and what you have imported into your IPython session:

```
In [1]: import math

In [2]: ma<Tab>
           map          %macro       %%markdown
           math         %magic       %matplotlib
           max()        %man
```

You can scroll through the identifiers with the up and down arrow keys. As you do, IPython highlights an identifier and shows it to the right of the `In []` prompt.

Viewing Identifiers in a Module

To view a list of identifiers defined in a module, type the module's name and a dot (`.`), then press *Tab*:

```
In [3]: math.<Tab>
         acos()      atan()      copysign()  e           expm1()
         acosh()     atan2()     cos()       erf()       fabs()
         asin()      atanh()     cosh()      erfc()      factorial() >
         asinh()     ceil()      degrees()   exp()       floor()
```

If there are more identifiers to display than are currently shown, IPython displays the > symbol (on some platforms) at the right edge, in this case to the right of `factorial()`. You can use the up and down arrow keys to scroll through the list. In the list of identifiers:

- Those followed by parentheses are functions (or methods, as you'll see later).
- Single-word identifiers (such as `Employee`) that begin with an uppercase letter and multiword identifiers in which each word begins with an uppercase letter (such as `CommissionEmployee`) represent class names (there are none in the preceding list). This naming convention, which the *Style Guide for Python Code* recommends, is known as **CamelCase** because the uppercase letters stand out like a camel's humps.
- Lowercase identifiers without parentheses, such as `pi` (not shown in the preceding list) and `e`, are variables. The identifier `pi` evaluates to `3.141592653589793`, and the identifier `e` evaluates to `2.718281828459045`. In the `math` module, `pi` and `e` represent the mathematical constants π and e, respectively.

Python does not have *constants*, although many objects in Python are immutable (nonmodifiable). So even though `pi` and `e` are real-world constants, *you must not assign new values to them*, because that would change their values. To help distinguish constants from other variables, the style guide recommends naming your custom constants with all capital letters.

Using the Currently Highlighted Function

As you navigate through the identifiers, if you wish to use a currently highlighted function, simply start typing its arguments in parentheses. IPython then hides the autocompletion list. If you need more information about the currently highlighted item, you can view its docstring by typing a question mark (`?`) following the name and pressing *Enter* to view the help documentation. The following shows the `fabs` function's docstring:

```
In [4]: math.fabs?
Docstring:
fabs(x)

Return the absolute value of the float x.
Type:      builtin_function_or_method
```

The `builtin_function_or_method` shown above indicates that `fabs` is part of a Python Standard Library module. Such modules are considered to be built into Python. In this case, `fabs` is a built-in function from the `math` module.

Self Check

1 *(True/False)* In IPython interactive mode, to view a list of identifiers defined in a module, type the module's name and a dot (`.`) then press *Enter*.
Answer: False. Press *Tab*, not *Enter*.

2 *(True/False)* Python does not have constants.
Answer: True.

4.9 Default Parameter Values

When defining a function, you can specify that a parameter has a **default parameter value**. When calling the function, if you omit the argument for a parameter with a default parameter value, the default value for that parameter is automatically passed. Let's define a function `rectangle_area` with default parameter values:

```
In [1]: def rectangle_area(length=2, width=3):
   ...:     """Return a rectangle's area."""
   ...:     return length * width
   ...:
```

You specify a default parameter value by following a parameter's name with an `=` and a value—in this case, the default parameter values are `2` and `3` for `length` and `width`, respectively. Any parameters with default parameter values must appear in the parameter list to the *right* of parameters that do not have defaults.

The following call to `rectangle_area` has no arguments, so IPython uses both default parameter values as if you had called `rectangle_area(2, 3)`:

```
In [2]: rectangle_area()
Out[2]: 6
```

The following call to `rectangle_area` has only one argument. Arguments are assigned to parameters from left to right, so `10` is used as the `length`. The interpreter passes the default parameter value 3 for the `width` as if you had called `rectangle_area(10, 3)`:

```
In [3]: rectangle_area(10)
Out[3]: 30
```

The following call to `rectangle_area` has arguments for both `length` and `width`, so IPython ignores the default parameter values:

```
In [4]: rectangle_area(10, 5)
Out[4]: 50
```

Self Check

1 *(True/False)* When an argument with a default parameter value is omitted in a function call, the interpreter automatically passes the default parameter value in the call.
Answer: True.

2 *(True/False)* Parameters with default parameter values must be the *leftmost* arguments in a function's parameter list.
Answer: False. Parameters with default parameter values must appear to the *right* of parameters that do not have defaults.

4.10 Keyword Arguments

When calling functions, you can use **keyword arguments** to pass arguments in *any* order. To demonstrate keyword arguments, we redefine the `rectangle_area` function—this time without default parameter values:

```
In [1]: def rectangle_area(length, width):
   ...:     """Return a rectangle's area."""
   ...:     return length * width
   ...:
```

Each keyword *argument in a call* has the form *parametername=value*. The following call shows that the order of keyword arguments does not matter—they do not need to match the corresponding parameters' positions in the function definition:

```
In [2]: rectangle_area(width=5, length=10)
Out[3]: 50
```

In each function call, you must place keyword arguments *after* a function's positional arguments—that is, any arguments for which you do not specify the parameter name. Such arguments are assigned to the function's parameters left-to-right, based on the argument's positions in the argument list. Keyword arguments are also helpful for improving the readability of function calls, especially for functions with many arguments.

Self Check

1 *(True/False)* You must pass keyword arguments in the same order as their corresponding parameters in the function definition's parameter list.
Answer: False. The order of keyword arguments does not matter.

4.11 Arbitrary Argument Lists

Functions with **arbitrary argument lists**, such as built-in functions `min` and `max`, can receive *any* number of arguments. Consider the following `min` call:

```
min(88, 75, 96, 55, 83)
```

The function's documentation states that `min` has two *required* parameters (named `arg1` and `arg2`) and an optional third parameter of the form **`*args`**, indicating that the function can receive any number of additional arguments. The `*` before the parameter name tells Python to pack any remaining arguments into a tuple that's passed to the `args` parameter. In the call above, parameter `arg1` receives `88`, parameter `arg2` receives `75` and parameter `args` receives the tuple `(96, 55, 83)`.

Defining a Function with an Arbitrary Argument List

Let's define an `average` function that can receive any number of arguments:

```
In [1]: def average(*args):
   ...:     return sum(args) / len(args)
   ...:
```

The parameter name `args` is used by convention, but you may use any identifier. If the function has multiple parameters, the `*args` parameter must be the *rightmost* parameter.

Now, let's call `average` several times with arbitrary argument lists of different lengths:

```
In [2]: average(5, 10)
Out[2]: 7.5

In [3]: average(5, 10, 15)
Out[3]: 10.0

In [4]: average(5, 10, 15, 20)
Out[4]: 12.5
```

To calculate the average, divide the sum of the `args` tuple's elements (returned by built-in function `sum`) by the tuple's number of elements (returned by built-in function `len`). Note in our `average` definition that if the length of `args` is 0, a `ZeroDivisionError` occurs. In the next chapter, you'll see how to access a tuple's elements without unpacking them.

Passing an Iterable's Individual Elements as Function Arguments

You can unpack a tuple's, list's or other iterable's elements to pass them as individual function arguments. The **`*` operator**, when applied to an iterable argument in a function call, unpacks its elements. The following code creates a five-element `grades` list, then uses the expression `*grades` to unpack its elements as `average`'s arguments:

```
In [5]: grades = [88, 75, 96, 55, 83]

In [6]: average(*grades)
Out[6]: 79.4
```

The call shown above is equivalent to `average(88, 75, 96, 55, 83)`.

Self Check

1 *(Fill-In)* To define a function with an arbitrary argument list, specify a parameter of the form _____.

Answer: `*args` (again, the name `args` is used by convention, but is not required).

2 *(IPython Session)* Create a function named `calculate_product` that receives an arbitrary argument list and returns the product of all the arguments. Call the function with the arguments `10`, `20` and `30`, then with the sequence of integers produced by `range(1, 6, 2)`.

Answer:

```
In [1]: def calculate_product(*args):
   ...:     product = 1
   ...:     for value in args:
   ...:         product *= value
   ...:     return product
   ...:
```

```
In [2]: calculate_product(10, 20, 30)
Out[2]: 6000

In [3]: calculate_product(*range(1, 6, 2))
Out[3]: 15
```

4.12 Methods: Functions That Belong to Objects

A **method** is simply a function that you call on an object using the form

object_name.*method_name*(*arguments*)

For example, the following session creates the string variable `s` and assigns it the string object `'Hello'`. Then the session calls the object's **`lower`** and **`upper`** methods, which produce *new* strings containing all-lowercase and all-uppercase versions of the original string, leaving `s` unchanged:

```
In [1]: s = 'Hello'

In [2]: s.lower()  # call lower method on string object s
Out[2]: 'hello'

In [3]: s.upper()
Out[3]: 'HELLO'

In [4]: s
Out[4]: 'Hello'
```

The *Python Standard Library* reference at

`https://docs.python.org/3/library/index.html`

describes the methods of built-in types and the types in the Python Standard Library. In the "Object-Oriented Programming" chapter, you'll create *custom* types called classes and define custom methods that you can call on objects of those classes.

4.13 Scope Rules

Each identifier has a **scope** that determines where you can use it in your program. For that portion of the program, the identifier is said to be "in scope."

Local Scope

A local variable's identifier has **local scope**. It's "in scope" only from its definition to the end of the function's block. It "goes out of scope" when the function returns to its caller. So, a local variable can be used only inside the function that defines it.

Global Scope

Identifiers defined outside any function (or class) have **global scope**—these may include functions, variables and classes. Variables with global scope are known as **global variables**. Identifiers with global scope can be used in a `.py` file or interactive session anywhere after they're defined.

Accessing a Global Variable from a Function

You can access a global variable's value inside a function:

```
In [1]: x = 7

In [2]: def access_global():
   ...:     print('x printed from access_global:', x)
   ...:

In [3]: access_global()
x printed from access_global: 7
```

However, by default, you cannot *modify* a global variable in a function—when you first assign a value to a variable in a function's block, Python creates a *new* local variable:

```
In [4]: def try_to_modify_global():
   ...:     x = 3.5
   ...:     print('x printed from try_to_modify_global:', x)
   ...:

In [5]: try_to_modify_global()
x printed from try_to_modify_global: 3.5

In [6]: x
Out[6]: 7
```

In function `try_to_modify_global`'s block, the local `x` **shadows** the global `x`, making it inaccessible in the scope of the function's block. Snippet `[6]` shows that global variable `x` still exists and has its original value (7) after function `try_to_modify_global` executes.

To modify a global variable in a function's block, you must use a **`global`** statement to declare that the variable is defined in the global scope:

```
In [7]: def modify_global():
   ...:     global x
   ...:     x = 'hello'
   ...:     print('x printed from modify_global:', x)
   ...:

In [8]: modify_global()
x printed from modify_global: hello

In [9]: x
Out[9]: 'hello'
```

Blocks vs. Suites

You've now defined function *blocks* and control statement *suites*. When you create a variable in a block, it's *local* to that block. However, when you create a variable in a control statement's suite, the variable's scope depends on where the control statement is defined:

- If the control statement is in the global scope, then any variables defined in the control statement have global scope.
- If the control statement is in a function's block, then any variables defined in the control statement have local scope.

We'll continue our scope discussion in the "Object-Oriented Programming" chapter when we introduce custom classes.

Shadowing Functions

In the preceding chapters, when summing values, we stored the sum in a variable named `total`. The reason we did this is that `sum` is a built-in function. If you define a variable named `sum`, it *shadows* the built-in function, making it inaccessible in your code. When you execute the following assignment, Python binds the identifier `sum` to the `int` object containing `15`. At this point, the identifier `sum` no longer references the built-in function. So, when you try to use `sum` as a function, a `TypeError` occurs:

```
In [10]: sum = 10 + 5

In [11]: sum
Out[11]: 15

In [12]: sum([10, 5])
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-12-1237d97a65fb> in <module>()
----> 1 sum([10, 5])

TypeError: 'int' object is not callable
```

Statements at Global Scope

In the scripts you've seen so far, we've written some statements outside functions at the global scope and some statements inside function blocks. Script statements at global scope execute as soon as they're encountered by the interpreter, whereas statements in a block execute only when the function is called.

Self Check

1 *(Fill-In)* An identifier's _____ describes the region of a program in which the identifier's value can be accessed.
Answer: scope.

2 *(True/False)* Once a code block terminates (e.g., when a function returns), all identifiers defined in that block "go out of scope" and can no longer be accessed.
Answer: True.

4.14 `import`: A Deeper Look

You've imported modules (such as `math` and `random`) with a statement like:

```
import module_name
```

then accessed their features via each module's name and a dot (`.`). Also, you've imported a specific identifier from a module (such as the `decimal` module's `Decimal` type) with a statement like:

```
from module_name import identifier
```

then used that identifier without having to precede it with the module name and a dot (`.`).

Importing Multiple Identifiers from a Module

Using the `from...import` statement you can import a comma-separated list of identifiers from a module then use them in your code without having to precede them with the module name and a dot (`.`):

```
In [1]: from math import ceil, floor

In [2]: ceil(10.3)
Out[2]: 11

In [3]: floor(10.7)
Out[3]: 10
```

Trying to use a function that's not imported causes a `NameError`, indicating that the name is not defined.

Caution: Avoid Wildcard Imports

You can import *all* identifiers defined in a module with a **wildcard import** of the form

```
from modulename import *
```

This makes all of the module's identifiers available for use in your code. Importing a module's identifiers with a wildcard `import` can lead to subtle errors—it's considered a dangerous practice that you should avoid. Consider the following snippets:

```
In [4]: e = 'hello'

In [5]: from math import *

In [6]: e
Out[6]: 2.718281828459045
```

Initially, we assign the string `'hello'` to a variable named `e`. After executing snippet `[5]` though, the variable `e` is replaced, possibly by accident, with the `math` module's constant `e`, representing the mathematical floating-point value *e*.

Binding Names for Modules and Module Identifiers

Sometimes it's helpful to import a module and use an abbreviation for it to simplify your code. The `import` statement's **`as`** clause allows you to specify the name used to reference the module's identifiers. For example, in Section 3.17 we could have imported the `statistics` module and accessed its `mean` function as follows:

```
In [7]: import statistics as stats

In [8]: grades = [85, 93, 45, 87, 93]

In [9]: stats.mean(grades)
Out[9]: 80.6
```

As you'll see in later chapters, `import...as` is frequently used to import Python libraries with convenient abbreviations, like `stats` for the `statistics` module. As another example, we'll use the `numpy` module which typically is imported with

```
import numpy as np
```

Library documentation often mentions popular shorthand names.

Typically, when importing a module, you should use `import` or `import...as` statements, then access the module through the module name or the abbreviation following the `as` keyword, respectively. This ensures that you do not accidentally import an identifier that conflicts with one in your code.

Self Check

1 *(True/False)* You must always import all the identifiers of a given module.
Answer: False. You can import only the identifiers you need by using a `from...import` statement.

2 *(IPython Session)* Import the `decimal` module with the shorthand name `dec`, then create a `Decimal` object with the value `2.5` and square its value.
Answer:

```
In [1]: import decimal as dec

In [2]: dec.Decimal('2.5') ** 2
Out[2]: Decimal('6.25')
```

4.15 Passing Arguments to Functions: A Deeper Look

Let's take a closer look at how arguments are passed to functions. In many programming languages, there are two ways to pass arguments—**pass-by-value** and **pass-by-reference** (sometimes called **call-by-value** and **call-by-reference**, respectively):

- With pass-by-value, the called function receives a *copy* of the argument's *value* and works exclusively with that copy. Changes to the function's copy do *not* affect the original variable's value in the caller.
- With pass-by-reference, the called function can access the argument's value in the caller directly and modify the value if it's mutable.

Python arguments are always passed by reference. Some people call this **pass-by-object-reference**, because "everything in Python is an object."[3] When a function call provides an argument, Python copies the argument object's *reference*—not the object itself—into the corresponding parameter. This is important for performance. Functions often manipulate large objects—frequently copying them would consume large amounts of computer memory and significantly slow program performance.

Memory Addresses, References and "Pointers"

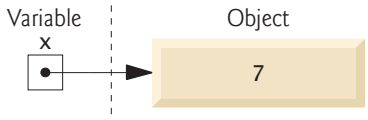
You interact with an object via a reference, which behind the scenes is that object's address (or location) in the computer's memory—sometimes called a "pointer" in other languages. After an assignment like

```
x = 7
```

the variable `x` does not actually contain the value `7`. Rather, it contains a reference to an *object* containing `7` (and some other data we'll discuss in later chapters) stored *elsewhere* in

3. Even the functions you defined in this chapter and the classes (custom types) you'll define in later chapters are objects in Python.

memory. You might say that x "points to" (that is, references) the object containing 7, as in the diagram below:



Built-In Function id and Object Identities

Let's consider how we pass arguments to functions. First, let's create the integer variable x mentioned above—shortly we'll use x as a function argument:

```
In [1]: x = 7
```

Now x refers to (or "points to") the integer object containing 7. No two separate objects can reside at the same address in memory, so every object in memory has a *unique address*. Though we can't see an object's address, we can use the built-in **id function** to obtain a *unique* int value which identifies only that object while it remains in memory (you'll likely get a different value when you run this on your computer):

```
In [2]: id(x)
Out[2]: 4350477840
```

The integer result of calling id is known as the object's **identity**.[4] No two objects in memory can have the same *identity*. We'll use object identities to demonstrate that objects are passed by reference.

Passing an Object to a Function

Let's define a cube function that displays its parameter's identity, then returns the parameter's value cubed:

```
In [3]: def cube(number):
   ...:     print('id(number):', id(number))
   ...:     return number ** 3
   ...:
```

Next, let's call cube with the argument x, which refers to the integer object containing 7:

```
In [4]: cube(x)
id(number): 4350477840
Out[4]: 343
```

The identity displayed for cube's parameter number—4350477840—is the *same* as that displayed for x previously. Since every object has a unique identity, both the *argument* x and the *parameter* number refer to the *same object* while cube executes. So when function cube uses its parameter number in its calculation, it gets the value of number from the original object in the caller.

Testing Object Identities with the is Operator

You also can prove that the argument and the parameter refer to the same object with Python's **is operator**, which returns True if its two operands have the *same identity*:

4. According to the Python documentation, depending on the Python implementation you're using, an object's identity may be the object's actual memory address, but this is not required.

```
In [5]: def cube(number):
   ...:     print('number is x:', number is x)  # x is a global variable
   ...:     return number ** 3
   ...:

In [6]: cube(x)
number is x: True
Out[6]: 343
```

Immutable Objects as Arguments

When a function receives as an argument a reference to an *immutable* (unmodifiable) object—such as an `int`, `float`, `string` or `tuple`—even though you have direct access to the original object in the caller, you cannot modify the original immutable object's value. To prove this, first let's have `cube` display `id(number)` before and after assigning a new object to the parameter `number` via an augmented assignment:

```
In [7]: def cube(number):
   ...:     print('id(number) before modifying number:', id(number))
   ...:     number **= 3
   ...:     print('id(number) after modifying number:', id(number))
   ...:     return number
   ...:

In [8]: cube(x)
id(number) before modifying number: 4350477840
id(number) after modifying number: 4396653744
Out[8]: 343
```

When we call `cube(x)`, the first `print` statement shows that `id(number)` initially is the same as `id(x)` in snippet `[2]`. Numeric values are immutable, so the statement

```
number **= 3
```

actually creates a *new object* containing the cubed value, then assigns that object's reference to parameter `number`. Recall that if there are no more references to the *original* object, it will be *garbage collected*. Function `cube`'s second `print` statement shows the *new* object's identity. Object identities must be unique, so `number` must refer to a *different* object. To show that `x` was not modified, we display its value and identity again:

```
In [9]: print(f'x = {x}; id(x) = {id(x)}')
x = 7; id(x) = 4350477840
```

Mutable Objects as Arguments

In the next chapter, we'll show that when a reference to a *mutable* object like a list is passed to a function, the function *can* modify the original object in the caller.

Self Check

1 *(Fill-In)* The built-in function _____ returns an object's unique identifier.
Answer: `id`.

2 *(True/False)* Attempts to modify mutable objects create new objects.
Answer: False. This is true for immutable objects.

3 *(IPython Session)* Create a variable `width` with the value `15.5`, then show that modifying the variable creates a new object. Display `width`'s identity and value before and after modifying its value.
Answer:

```
In [1]: width = 15.5

In [2]: print('id:', id(width), ' value:', width)
id: 4397553776  value: 15.5

In [3]: width = width * 3

In [4]: print('id:', id(width), ' value:', width)
id: 4397554208  value: 46.5
```

4.16 Function-Call Stack

To understand how Python performs function calls, consider a data structure (that is, a collection of related data items) known as a **stack**, which is like a pile of dishes. When you add a dish to the pile, you place it on the *top*. Similarly, when you remove a dish from the pile, you take it from the top. Stacks are known as **last-in, first-out (LIFO) data structures**—the last item **pushed** (that is, placed) onto the stack is the first item **popped** (that, is removed) from the stack.

Stacks and Your Web Browser's Back Button

A stack is working for you when you visit websites with your web browser. A stack of webpage addresses supports a browser's back button. For each new web page you visit, the browser *pushes* the address of the page you were viewing onto the back button's stack. This allows the browser to "remember" the web page you came from if you decide to go back to it later. Pushing onto the back button's stack may happen many times before you decide to go back to a previous web page. When you press the browser's back button, the browser *pops* the top stack element to get the prior web page's address, then displays that web page. Each time you press the back button the browser *pops* the top stack element and displays that page. This continues until the stack is empty, meaning that there are no more pages for you to go back to via the back button.

Stack Frames

Similarly, the **function-call stack** supports the function call/return mechanism. Eventually, each function must return program control to the point at which it was called. For each function call, the interpreter *pushes* an entry called a **stack frame** (or an **activation record**) onto the stack. This entry contains the *return location* that the called function needs so it can return control to its caller. When the function finishes executing, the interpreter *pops* the function's stack frame, and control transfers to the *return location* that was popped.

The *top* stack frame *always* contains the information the currently executing function needs to return control to its caller. If before a function returns it makes a call to another function, the interpreter *pushes* a stack frame for that function call onto the stack. Thus, the return address required by the newly called function to return to its caller is now on *top* of the stack.

Local Variables and Stack Frames

Most functions have one or more *parameters* and possibly *local variables* that need to:

- exist while the function is executing,
- remain active if the function makes calls to other functions, and
- "go away" when the function returns to its caller.

A called function's stack frame is the perfect place to reserve memory for the function's local variables. That stack frame is pushed when the function is called and exists while the function is executing. When that function returns, it no longer needs its local variables, so its stack frame is *popped* from the stack, and its local variables no longer exist.

Stack Overflow

Of course, the amount of memory in a computer is finite, so only a certain amount of memory can be used to store stack frames on the function-call stack. If the function-call stack runs out of memory as a result of too many function calls, a fatal error known as **stack overflow** occurs.[5] Stack overflows actually are rare unless you have a logic error that keeps calling functions that never return.

Principle of Least Privilege

The **principle of least privilege** is fundamental to good software engineering. It states that code should be granted *only* the amount of privilege and access that it needs to accomplish its designated task, but no more. An example of this is the scope of a local variable, which should not be visible when it's not needed. This is why a function's local variables are placed in stack frames on the function-call stack, so they can be used by that function while it executes and go away when it returns. Once the stack frame is popped, the memory that was occupied by it can be reused for new stack frames. Also, there is no access between stack frames, so functions cannot see each other's local variables. The principle of least privilege makes your programs more robust by preventing code from accidentally (or maliciously) modifying variable values that should not be accessible to it.

Self Check

1 *(Fill-In)* The stack operations for adding an item to a stack and removing an item from a stack are known as _____ and _____, respectively.
Answer: push, pop.

2 *(Fill-In)* A stack's items are removed in _____ order.
Answer: last-in, first-out (LIFO).

4.17 Functional-Style Programming

Like other popular languages, such as Java and C#, Python is not a purely functional language. Rather, it offers "functional-style" features that help you write code which is less likely to contain errors, more concise and easier to read, debug and modify. Functional-style programs also can be easier to parallelize to get better performance on today's multi-

5. This is how the website `stackoverflow.com` got its name—a good website for getting answers to your programming questions.

core processors. The chart below lists most of Python's key functional-style programming capabilities and shows in parentheses the chapters in which we initially cover many of them.

Functional-style programming topics		
avoiding side effects	generator functions (12)	lazy evaluation (5)
closures	higher-order functions (5)	list comprehensions (5)
declarative programming (4)	immutability (4)	`operator` module (5, 13, 18)
decorators (10)	internal iteration (4)	pure functions (4)
dictionary comprehensions (6)	iterators (3)	`range` function (3, 4)
`filter`/`map`/`reduce` (5)	`itertools` module (18)	reductions (3, 5)
`functools` module	`lambda` expressions (5)	set comprehensions (6)
generator expressions (5)		

We cover most of these features throughout the book—many with code examples and others from a literacy perspective. You've already used list, string and built-in function `range` *iterators* with the `for` statement, and several *reductions* (functions `sum`, `len`, `min` and `max`). We discuss declarative programming, immutability and internal iteration below.

What vs. How

As the tasks you perform get more complicated, your code can become harder to read, debug and modify, and more likely to contain errors. Specifying *how* the code works can become complex.

Functional-style programming lets you simply say *what* you want to do. It hides many details of *how* to perform each task. Typically, library code handles the *how* for you. As you'll see, this can eliminate many errors.

Consider the `for` statement in many other programming languages. Typically, *you* must specify all the details of counter-controlled iteration: a control variable, its initial value, how to increment it and a loop-continuation condition that uses the control variable to determine whether to continue iterating. This style of iteration is known as **external iteration** and is error-prone. For example, you might provide an incorrect initializer, increment or loop-continuation condition. External iteration **mutates** (that is, modifies) the control variable, and the `for` statement's suite often mutates other variables as well. Every time you modify variables you could introduce errors. Functional-style programming emphasizes **immutability**. That is, it avoids operations that modify variables' values. We'll say more in the next chapter.

Python's `for` statement and `range` function *hide* most counter-controlled iteration details. You specify *what* values `range` should produce and the variable that should receive each value as it's produced. Function `range` *knows how* to produce those values. Similarly, the `for` statement *knows how* to get each value from `range` and *how* to stop iterating when there are no more values. Specifying *what*, but not *how*, is an important aspect of **internal iteration**—a key functional-style programming concept.

The Python built-in functions `sum`, `min` and `max` each use internal iteration. To total the elements of the list `grades`, you simply declare *what* you want to do—that is, `sum(grades)`. Function `sum` *knows how* to iterate through the list and add each element to

the running total. Stating what you *want* done rather than programming *how* to do it is known as **declarative programming**.

Pure Functions

In pure functional programming language you focus on writing pure functions. A **pure function's** result depends only on the argument(s) you pass to it. Also, given a particular argument (or arguments), a pure function always produces the same result. For example, built-in function `sum`'s return value depends only on the iterable you pass to it. Given a list `[1, 2, 3]`, `sum` *always* returns 6 no matter how many times you call it. Also, a pure function does not have *side effects*. For example, even if you pass a *mutable* list to a pure function, the list will contain the same values before and after the function call. When you call the pure function `sum`, it does not modify its argument.

```
In [1]: values = [1, 2, 3]

In [2]: sum(values)
Out[2]: 6

In [3]: sum(values)  # same call always returns same result
Out[3]: 6

In [4]: values
Out[5]: [1, 2, 3]
```

In the next chapter, we'll continue using functional-style programming concepts. Also, you'll see that *functions are objects* that you can pass to other functions as data.

4.18 Intro to Data Science: Measures of Dispersion

In our discussion of descriptive statistics, we've considered the measures of central tendency—mean, median and mode. These help us categorize typical values in a group—such as the mean height of your classmates or the most frequently purchased car brand (the mode) in a given country.

When we're talking about a group, the entire group is called the **population**. Sometimes a population is quite large, such as the people likely to vote in the next U.S. presidential election, which is a number in excess of 100,000,000 people. For practical reasons, the polling organizations trying to predict who will become the next president work with carefully selected small subsets of the population known as **samples**. Many of the polls in the 2016 election had sample sizes of about 1000 people.

In this section, we continue discussing basic descriptive statistics. We introduce **measures of dispersion** (also called **measures of variability**) that help you understand how spread out the values are. For example, in a class of students, there may be a bunch of students whose height is close to the average, with smaller numbers of students who are considerably shorter or taller.

For our purposes, we'll calculate each measure of dispersion both by hand and with functions from the module `statistics`, using the following population of 10 six-sided die rolls:

```
1, 3, 4, 2, 6, 5, 3, 4, 5, 2
```

Variance

To determine the **variance**,[6] we begin with the mean of these values—`3.5`. You obtain this result by dividing the sum of the face values, `35`, by the number of rolls, `10`. Next, we subtract the mean from every die value (this produces some negative results):

```
-2.5, -0.5, 0.5, -1.5, 2.5, 1.5, -0.5, 0.5, 1.5, -1.5
```

Then, we square each of these results (yielding only positives):

```
6.25, 0.25, 0.25, 2.25, 6.25, 2.25, 0.25, 0.25, 2.25, 2.25
```

Finally, we calculate the mean of these squares, which is `2.25` (`22.5 / 10`)—this is the **population variance**. Squaring the difference between each die value and the mean of all die values emphasizes **outliers**—the values that are farthest from the mean. As we get deeper into data analytics, sometimes we'll want to pay careful attention to outliers, and sometimes we'll want to ignore them. The following code uses the `statistics` module's **`pvariance`** function to confirm our manual result:

```
In [1]: import statistics

In [2]: statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[2]: 2.25
```

Standard Deviation

The **standard deviation** is the square root of the variance (in this case, `1.5`), which tones down the effect of the outliers. The smaller the variance and standard deviation are, the closer the data values are to the mean and the less overall **dispersion** (that is, **spread**) there is between the values and the mean. The following code calculates the **population standard deviation** with the statistics module's **`pstdev`** function, confirming our manual result:

```
In [3]: statistics.pstdev([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[3]: 1.5
```

Passing the `pvariance` function's result to the `math` module's `sqrt` function confirms our result of `1.5`:

```
In [4]: import math

In [5]: math.sqrt(statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2]))
Out[5]: 1.5
```

Advantage of Population Standard Deviation vs. Population Variance

Suppose you've recorded the March Fahrenheit temperatures in your area. You might have 31 numbers such as 19, 32, 28 and 35. The units for these numbers are degrees. When you square your temperatures to calculate the population variance, the units of the population variance become "degrees squared." When you take the square root of the popula-

6. For simplicity, we're calculating the *population variance*. There is a subtle difference between the *population variance* and the *sample variance*. Instead of dividing by n (the number of die rolls in our example), sample variance divides by $n - 1$. The difference is pronounced for small samples and becomes insignificant as the sample size increases. The `statistics` module provides the functions `pvariance` and `variance` to calculate the population variance and sample variance, respectively. Similarly, the `statistics` module provides the functions `pstdev` and `stdev` to calculate the population standard deviation and sample standard deviation, respectively.

tion variance to calculate the population standard deviation, the units once again become degrees, which are the *same* units as your temperatures.

Self Check

1 *(Discussion)* Why do we often work with a sample rather than the full population?
Answer: Because often the full population is unmanageably large.

2 *(True/False)* An advantage of the population variance over the population standard deviation is that its units are the same as the sample values' units.
Answer: False. This an advantage of population standard deviation over population variance.

3 *(IPython Session)* In this section, we worked with population variance and population standard deviation. There is a subtle difference between the *population variance* and the *sample variance*. In our example, instead of dividing by 10 (the number of die rolls), sample variance would divide by 9 (which is one less than the sample size). The difference is pronounced for small samples but becomes insignificant as the sample size increases. The `statistics` module provides the functions `variance` and `stdev` to calculate the sample variance and sample standard deviation, respectively. Redo the manual calculations, then use the `statistics` module's functions to confirm this difference between the two methods of calculation.
Answer:

```
In [1]: import statistics

In [2]: statistics.variance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[2]: 2.5

In [3]: statistics.stdev([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[3]: 1.5811388300841898
```

4.19 Wrap-Up

In this chapter, we created custom functions. We imported capabilities from the `random` and `math` modules. We introduced random-number generation and used it to simulate rolling a six-sided die. We packed multiple values into tuples to return more than one value from a function. We also unpacked a tuple to access its values. We discussed using the Python Standard Library's modules to avoid "reinventing the wheel."

We created functions with default parameter values and called functions with keyword arguments. We also defined functions with arbitrary argument lists. We called methods of objects. We discussed how an identifier's scope determines where in your program you can use it.

You learned more about importing modules. You saw that arguments are passed-by-reference to functions, and how the function-call stack and stack frames support the function-call-and-return mechanism. We've introduced basic list and tuple capabilities over the last two chapters—in the next chapter, we'll discuss them in detail.

Finally, we continued our discussion of descriptive statistics by introducing measures of dispersion—variance and standard deviation—and calculating them with functions from the Python Standard Library's `statistics` module.

For some types of problems, it's useful to have functions call themselves. A **recursive function** calls itself, either directly or indirectly through another function. Recursion is an important topic discussed at length in upper-level computer science courses. We include a detailed treatment in the chapter "Computer Science Thinking: Recursion, Searching, Sorting and Big O."

Exercises

Unless specified otherwise, use IPython sessions for each exercise.

4.1 *(Discussion: `else` Clause)* In the script of Fig. 4.1, we did not include an `else` clause in the `if...elif` statement. What are the possible consequences of this choice?

4.2 *(Discussion: Function-Call Stack)* What happens if you keep pushing onto a stack, without enough popping?

4.3 *(What's Wrong with This Code?)* What is wrong with the following `cube` function's definition?

```
def cube(x):
    """Calculate the cube of x."""
    x ** 3

print('The cube of 2 is', cube(2))
```

4.4 *(What's Does This Code Do?)* What does the following `mystery` function do? Assume you pass the list `[1, 2, 3, 4, 5]` as an argument.

```
def mystery(x):
    y = 0

    for value in x:
        y += value ** 2

    return y
```

4.5 *(Fill in the Missing Code?)* Replace the ***s in the `seconds_since_midnight` function so that it returns the number of seconds since midnight. The function should receive three integers representing the current time of day. Assume that the hour is a value from `0` (midnight) through `23` (11 PM) and that the minute and second are values from `0` to `59`. Test your function with actual times. For example, if you call the function for 1:30:45 PM by passing `13`, `30` and `45`, the function should return `48645`.

```
def seconds_since_midnight(***):
    hour_in_seconds = ***
    minute_in_seconds = ***
    return ***
```

4.6 *(Modified `average` Function)* The `average` function we defined in Section 4.11 can receive any number of arguments. If you call it with no arguments, however, the function causes a `ZeroDivisionError`. Reimplement `average` to receive one required argument *and* the arbitrary argument list argument `*args`, and update its calculation accordingly. Test your function. The function will always require at least one argument, so you'll no longer be able to get a `ZeroDivisionError`. When you call `average` with no arguments, Python should issue a `TypeError` indicating `"average() missing 1 required positional argument."`

4.7 *(Date and Time)* Python's `datetime` module contains a `datetime` type with a method `today` that returns the current date and time as a `datetime` object. Write a *parameterless* `date_and_time` function containing the following statement, then call that function to display the current date and time:

```
print(datetime.datetime.today())
```

On our system, the date and time display in the following format:

```
2018-06-08 13:04:19.214180
```

4.8 *(Rounding Numbers)* Investigate built-in function `round` at

```
https://docs.python.org/3/library/functions.html#round
```

then use it to round the `float` value `13.56449` to the nearest integer, tenths, hundredths and thousandths positions.

4.9 *(Temperature Conversion)* Implement a `fahrenheit` function that returns the Fahrenheit equivalent of a Celsius temperature. Use the following formula:

```
F = (9 / 5) * C + 32
```

Use this function to print a chart showing the Fahrenheit equivalents of all Celsius temperatures in the range 0–100 degrees. Use one digit of precision for the results. Print the outputs in a neat tabular format.

4.10 *(Guess the Number)* Write a script that plays "guess the number." Choose the number to be guessed by selecting a random integer in the range 1 to 1000. Do not reveal this number to the user. Display the prompt `"Guess my number between 1 and 1000 with the fewest guesses:"`. The player inputs a first guess. If the guess is incorrect, display `"Too high. Try again."` or `"Too low. Try again."` as appropriate to help the player "zero in" on the correct answer, then prompt the user for the next guess. When the user enters the correct answer, display `"Congratulations. You guessed the number!"`, and allow the user to choose whether to play again.

4.11 *(Guess-the-Number Modification)* Modify the previous exercise to count the number of guesses the player makes. If the number is 10 or fewer, display `"Either you know the secret or you got lucky!"` If the player makes more than 10 guesses, display `"You should be able to do better!"` Why should it take no more than 10 guesses? Well, with each "good guess," the player should be able to eliminate half of the numbers, then half of the remaining numbers, and so on. Doing this 10 times narrows down the possibilities to a single number. This kind of "halving" appears in many computer science applications. For example, in the "Computer Science Thinking: Recursion, Searching, Sorting and Big O" chapter, we'll present the high-speed binary search and merge sort algorithms, and you'll attempt the quicksort exercise—each of these cleverly uses halving to achieve high performance.

4.12 *(Simulation: The Tortoise and the Hare)* In this problem, you'll re-create the classic race of the tortoise and the hare. You'll use random-number generation to develop a simulation of this memorable event.

Our contenders begin the race at square 1 of 70 squares. Each square represents a position along the race course. The finish line is at square 70. The first contender to reach or pass square 70 is rewarded with a pail of fresh carrots and lettuce. The course weaves its way up the side of a slippery mountain, so occasionally the contenders lose ground.

A clock ticks once per second. With each tick of the clock, your application should adjust the position of the animals according to the rules in the table below. Use variables to keep track of the positions of the animals (i.e., position numbers are 1–70). Start each animal at position 1 (the "starting gate"). If an animal slips left before square 1, move it back to square 1.

Animal	Move type	Percentage of the time	Actual move
Tortoise	Fast plod	50%	3 squares to the right
	Slip	20%	6 squares to the left
	Slow plod	30%	1 square to the right
Hare	Sleep	20%	No move at all
	Big hop	20%	9 squares to the right
	Big slip	10%	12 squares to the left
	Small hop	30%	1 square to the right
	Small slip	20%	2 squares to the left

Create two functions that generate the percentages in the table for the tortoise and the hare, respectively, by producing a random integer i in the range $1 \le i \le 10$. In the function for the tortoise, perform a "fast plod" when $1 \le i \le 5$, a "slip" when $6 \le i \le 7$ or a "slow plod" when $8 \le i \le 10$. Use a similar technique in the function for the hare.

Begin the race by displaying

```
BANG !!!!!
AND THEY'RE OFF !!!!!
```

Then, for each tick of the clock (i.e., each iteration of a loop), display a 70-position line showing the letter `"T"` in the position of the tortoise and the letter `"H"` in the position of the hare. Occasionally, the contenders will land on the same square. In this case, the tortoise bites the hare, and your application should display `"OUCH!!!"` at that position. All positions other than the `"T"`, the `"H"` or the `"OUCH!!!"` (in case of a tie) should be blank.

After each line is displayed, test for whether either animal has reached or passed square 70. If so, display the winner and terminate the simulation. If the tortoise wins, display `TORTOISE WINS!!! YAY!!!` If the hare wins, display `Hare wins. Yuch.` If both animals win on the same tick of the clock, you may want to favor the tortoise (the "underdog"), or you may want to display `"It's a tie"`. If neither animal wins, perform the loop again to simulate the next tick of the clock. When you're ready to run your application, assemble a group of fans to watch the race. You'll be amazed at how involved your audience gets!

4.13 ***(Arbitrary Argument List)*** Calculate the product of a series of integers that are passed to the function `product`, which receives an arbitrary argument list. Test your function with several calls, each with a different number of arguments.

4.14 ***(Computer-Assisted Instruction)*** *Computer-assisted instruction (CAI)* refers to the use of computers in education. Write a script to help an elementary school student learn multiplication. Create a function that randomly generates and returns a tuple of two pos-

itive one-digit integers. Use that function's result in your script to prompt the user with a question, such as

```
How much is 6 times 7?
```

For a correct answer, display the message `"Very good!"` and ask another multiplication question. For an incorrect answer, display the message `"No. Please try again."` and let the student try the same question repeatedly until the student finally gets it right.

4.15 ***(Computer-Assisted Instruction: Reducing Student Fatigue)*** Varying the computer's responses can help hold the student's attention. Modify the previous exercise so that various comments are displayed for each answer. Possible responses to a correct answer should include `'Very good!'`, `'Nice work!'` and `'Keep up the good work!'` Possible responses to an incorrect answer should include `'No. Please try again.'`, `'Wrong. Try once more.'` and `'No. Keep trying.'` Choose a number from 1 to 3, then use that value to select one of the three appropriate responses to each correct or incorrect answer.

4.16 ***(Computer-Assisted Instruction: Difficulty Levels)*** Modify the previous exercise to allow the user to enter a difficulty level. At a difficulty level of 1, the program should use only single-digit numbers in the problems and at a difficulty level of 2, numbers as large as two digits.

4.17 ***(Computer-Assisted Instruction: Varying the Types of Problems)*** Modify the previous exercise to allow the user to pick a type of arithmetic problem to study—`1` means addition problems only, `2` means subtraction problems only, `3` means multiplication problems only, `4` means division problems only (avoid dividing by 0) and `5` means a random mixture of all these types.

4.18 ***(Functional-Style Programming: Internal vs. External Iteration)*** Why is internal iteration preferable to external iteration in functional-style programming?

4.19 ***(Functional-Style Programming: What vs. How)*** Why is programming that emphasizes "what" preferable to programming that emphasizes "how"? What is it that makes "what" programming feasible?

4.20 ***(Intro to Data Science: Population Variance vs. Sample Variance)*** We mentioned in the Intro to Data Science section that there's a slight difference between the way the `statistics` module's functions calculate the population variance and the sample variance. The same is true for the population standard deviation and the sample standard deviation. Research the reason for these differences.

5

Sequences: Lists and Tuples

Objectives

In this chapter, you'll:

- Create and initialize lists and tuples.
- Refer to elements of lists, tuples and strings.
- Sort and search lists, and search tuples.
- Pass lists and tuples to functions and methods.
- Use list methods to perform common manipulations, such as searching for items, sorting a list, inserting items and removing items.
- Use additional Python functional-style programming capabilities, including lambdas and the functional-style programming operations filter, map and reduce.
- Use functional-style list comprehensions to create lists quickly and easily, and use generator expressions to generate values on demand.
- Use two-dimensional lists.
- Enhance your analysis and presentation skills with the Seaborn and Matplotlib visualization libraries.